Apostila do minicurso

Do Clique à API: UX de Verdade

22 problemas de experiência do usuário vistos dos dois lados, frontend e API, cada um na versão ruim e na versão boa.

Adrian Gustavo de Paiva Oliveira
Pedro Ferreira

Lojinha UX
o código das demos
github.com/adriangpo/lojinha-ux

Antes de começar

Como usar a apostila

Esta apostila acompanha o minicurso Do Clique à API: UX de Verdade, mas foi escrita para ser lida também por quem não estava na sala. São 22 temas, na ordem da aula, divididos em quatro blocos que seguem a vida de uma ação: antes do clique, durante a espera, quando dá errado e o que deixa tudo rápido.

Cada tema começa dizendo o que é e segue sempre a mesma ordem: o problema, contado com um caso da lojinha; por que ele acontece; a solução, com o título dizendo se ela mora no front ou na API; os cuidados; e um teste para fazer no seu projeto. A etiqueta acima do título diz onde o tema se resolve: Frontend, API ou Frontend + API. A metade dos problemas de experiência do usuário é decidida longe da tela.

A lojinha é a aplicação usada nas demonstrações: uma loja pequena, com uma página por tema e cada versão, ruim e boa, num arquivo próprio. O código completo está no repositório do QR da capa, e o capítulo final explica como rodar tudo na sua máquina. Os códigos desta apostila são versões curtas, feitas para explicar a ideia; os da lojinha são os completos.

Na lojinha, cada tema tem uma página própria, com as versões num seletor no canto superior direito.

As notas na margem ligam o texto à aula, à lojinha e a outros temas. As figuras são desenhos, não prints. Os quadros mostram a tela em momentos diferentes, como uma tira de quadrinhos. As linhas do tempo mostram as requisições como o monitor da aula: cada barra vai do envio até a resposta, e a faixa “na tela” mostra o que a pessoa está vendo. Nas duas, o vermelho marca o que deu errado.

No fim há um checklist com os 22 temas em forma de pergunta, para passar no seu projeto depois.

Do Clique à API

Sumário

Como usar a apostila 2

Os limites de tempo 4

Bloco 1 Antes do clique 5

1 Formulários e validação 6
2 Erros úteis 9
3 URL como estado 12

Bloco 2 Durante a espera 14

4 Estados da tela 15
5 Skeleton loading 17
6 Layout shift 19
7 Debounce e throttle 21
8 Race conditions e AbortController 24
9 Optimistic UI 27
10 Feedback de sucesso 29
11 Upload com progresso e tempo real 31

Bloco 3 Quando dá errado 34

12 Timeouts 35
13 Retries e exponential backoff 37
14 Clique duplo e idempotência 39
15 Rate limiting 42
16 Sessão expirada 44
17 Offline e conexão instável 46

Bloco 4 Rapidez de verdade 48

18 Caching 49
19 Imagens com signed URL e cache 52
20 Lazy loading 54
21 N+1 requests 56
22 Paginação e infinite scroll 58

Checklist para o seu projeto 61

Como rodar a lojinha 63

Referências 65

Antes de começar

Os limites de tempo

Quase todo tema desta apostila volta a três números. Eles foram resumidos por Jakob Nielsen em 1993, a partir de pesquisas mais antigas sobre atenção, e continuam valendo porque falam de gente, não de tecnologia: 0,1 segundo, 1 segundo e 10 segundos.

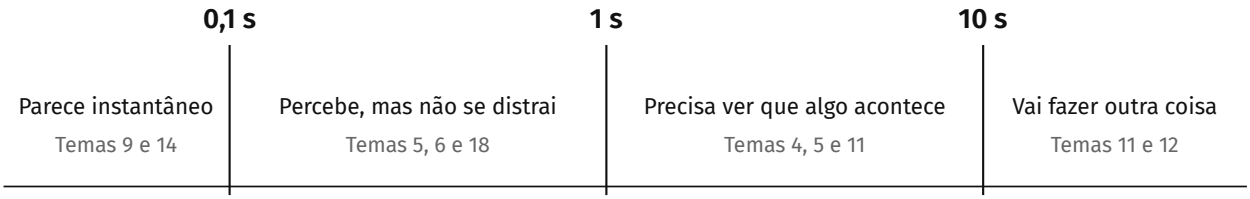


Figura 0.1. Os três limites e os temas em que cada um aparece.

Até 0,1 segundo, a resposta parece instantânea: a pessoa sente que o clique causou o efeito diretamente, sem nenhuma espera no meio. É o limite do coração que fica vermelho no Optimistic UI (tema 9) e do botão que muda para “Enviando...” no clique duplo (tema 14).

Até 1 segundo, a pessoa percebe que esperou, mas o raciocínio não se interrompe. Não precisa de aviso especial; precisa que a tela não fique em branco nem pule. É o território do skeleton (tema 5), do layout shift (tema 6) e do cache, que faz a volta a uma tela já vista ficar abaixo desse limite (tema 18, Caching).

Até 10 segundos, a atenção ainda se mantém na tarefa, desde que a tela mostre que algo está acontecendo. Passou disso, a pessoa vai fazer outra coisa, e a tela precisa mostrar progresso de verdade e avisar quando terminar. É o limite do upload com progresso (tema 11) e do timeout (tema 12), que existe justamente para a tela não ficar girando além dele.

Os números não são regras de engenharia. Eles servem para perguntar, diante de cada espera: em qual dessas faixas ela cai, e o que a tela está mostrando enquanto isso?

Bloco 1

Antes do clique

Antes de a pessoa clicar, a tela já pode evitar boa parte dos problemas: conferir o que foi digitado, deixar a API explicar o que recusou e guardar na URL o que a pessoa escolheu.

1	Formulários e validação	Frontend
2	Erros úteis	API
3	URL como estado	Frontend

Tema 1 · Frontend

Formulários e validação

Validar um formulário é conferir os dados antes de enviá-los e dizer à pessoa o que está faltando, no lugar certo e na hora certa. Uma parte dessa conferência só o servidor consegue fazer; o resto dá para resolver na própria tela, sem esperar resposta nenhuma.

1.1 O problema

O cadastro de produto da lojinha pede nome, categoria e preço. Na primeira versão, o botão Cadastrar está sempre liberado e o front não confere nada. A pessoa preenche só o nome, clica, espera a ida e a volta até a API e só então descobre que faltou a categoria. Se a mensagem vier genérica, ainda precisa adivinhar qual campo. E se o formulário for limpo depois do erro, digita tudo de novo.

Na lojinha, o tema tem três versões: Só no servidor, Botão sempre habilitado e Erro ao sair do campo.

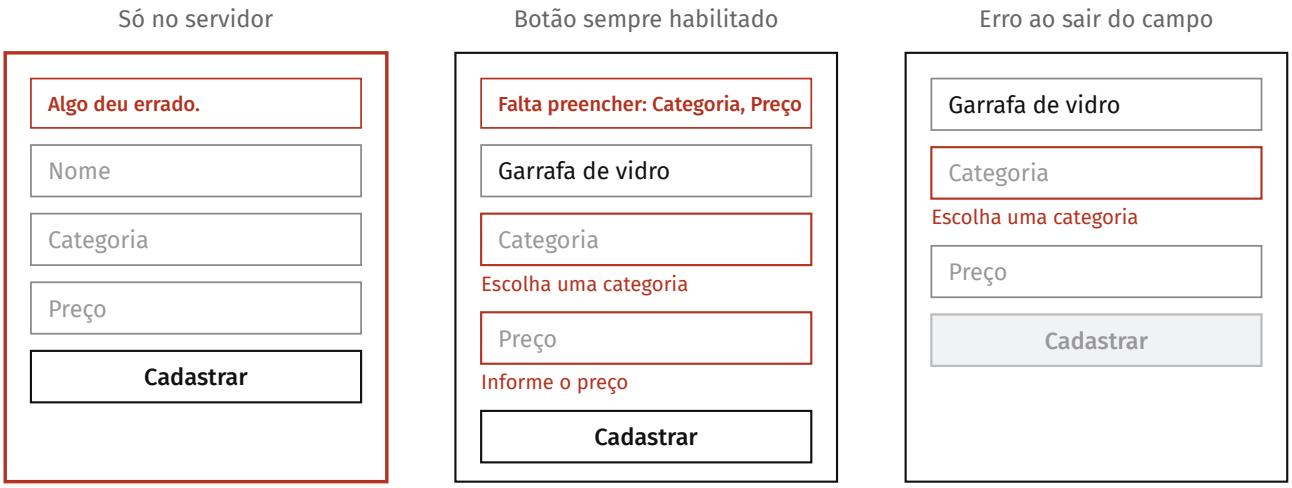


Figura 1.1. As três versões, da pior para a melhor. Nas duas primeiras, logo depois do clique em Cadastrar só com o nome; na última, quando a pessoa sai da categoria sem escolher.

1.2 Por que acontece

Quando toda a conferência fica no servidor, o erro só aparece depois de uma viagem pela rede, e a pessoa já tirou a atenção do formulário. Os dados para decidir se o nome está vazio ou se o preço é maior que zero já estão na tela; não há motivo para perguntar isso à API.

O outro motivo é o momento do aviso. Se os erros aparecem assim que a página abre, a pessoa vê o formulário em vermelho antes de digitar a primeira letra. Se aparecem só no envio, ela descobre tudo de uma vez,

no fim. O momento certo é quando ela termina um campo: entrou, saiu e deixou inválido. Campo em que ela ainda não chegou não mostra nada.

1.3 No front: avisar campo a campo

Os erros saem dos próprios valores, a cada mudança, e não precisam de estado próprio. O que precisa de estado é a lista de campos em que a pessoa já entrou e saiu, os *campos tocados* (*touched*, no vocabulário das bibliotecas de formulário): só eles mostram erro. Enquanto houver pendência, o botão fica com cara de desabilitado, mas continua clicável. O clique não envia: marca todos os campos como tocados, para os erros aparecerem de uma vez, e leva o foco ao primeiro que falta.

Frontend · Navegador

```
function errosDo(produto) {
  const erros = {}
  if (!produto.nome.trim()) erros.nome = 'Informe o nome do produto'
  if (!produto.categoria) erros.categoria = 'Escolha uma categoria'
  if (!(produto.preco > 0)) erros.preco = 'Informe um preço maior que zero'
  return erros
}

// Um campo só mostra erro depois que a pessoa entrou nele e saiu
campo.addEventListener('blur', () => { tocados.add(campo.name); mostrarErros() })

formulario.addEventListener('submit', (evento) => {
  evento.preventDefault()
  const pendentes = Object.keys(errosDo(lerProduto()))
  if (pendentes.length > 0) {            // clicou no botão cinza: não envia, explica
    pendentes.forEach((nome) => tocados.add(nome))
    mostrarErros()
    formulario.elements[pendentes[0]].focus()
    return
  }
  enviar()
})
```

Durante o envio, o botão troca para “Enviando...” e um segundo clique não faz nada. Se a API falhar, nada é apagado: a mensagem diz que os dados continuam ali e que dá para tentar de novo.

1.4 Cuidados

Desabilitar o botão de verdade, com o atributo `disabled`, tem críticos. A equipe do GOV.UK, o sistema de design do governo britânico, recomenda nunca desabilitar: um botão cinza não explica o que falta, e algumas pessoas nem percebem que ele está desligado. Por isso a última versão

Na aula, a turma votou entre o botão sempre habilitado e o desabilitado antes de ver a última versão, que junta os dois lados.

só parece desabilitada. Ela usa `aria-disabled`, continua acessível pelo teclado e, no clique, mostra o que falta; com `disabled`, o clique nem chegaria ao código. E nunca marque o campo pendente só com cor.

A mensagem que aparece ao sair do campo ocupa espaço. Sem reservar a altura dela, o erro surge e empurra o campo de baixo bem quando a pessoa vai clicar nele, e o clique erra o alvo. A versão da lojinha deixa cada campo com a altura de quem já tem erro, o mesmo cuidado do tema 6, Layout shift.

Nenhuma versão dispensa a validação do servidor. Qualquer um pode chamar a API sem passar pela tela, e algumas regras só o banco conhece, como um nome que já está em uso (tema 2, Erros úteis).

1.5 No seu projeto

Abra um formulário e passe pelos campos com Tab, sem digitar nada. Os erros aparecem só nos campos por onde você passou, em todos ao mesmo tempo ou em nenhum? Depois preencha pela metade e clique em enviar com a aba Network aberta: se saiu uma requisição, a conferência está só no servidor.

Tema 2 · API

Erros úteis

Quando a API recusa uma requisição, a resposta precisa dizer duas coisas: o tipo de problema, pelo status code, e o que exatamente está errado, num formato que o front consiga usar sem ler a frase. Com um código fixo para cada erro e o nome do campo, a tela põe a mensagem certa embaixo do campo certo.

2.1 O problema

A pessoa tenta cadastrar um produto chamado Teclado mecânico. Esse nome já existe, e só o banco sabe disso. Na versão ruim, a API responde `500` com `{"erro": "invalid"}`, e o front, sem saber o que fazer com isso, mostra “Algo deu errado”. A pessoa não sabe se foi a internet, se o servidor caiu ou se foi algo que ela fez. Tenta de novo, do mesmo jeito, e recebe o mesmo erro.

Na lojinha, o tema tem duas versões: Erro genérico e Erro com code por campo. O nome “Teclado mecânico” já existe no banco.

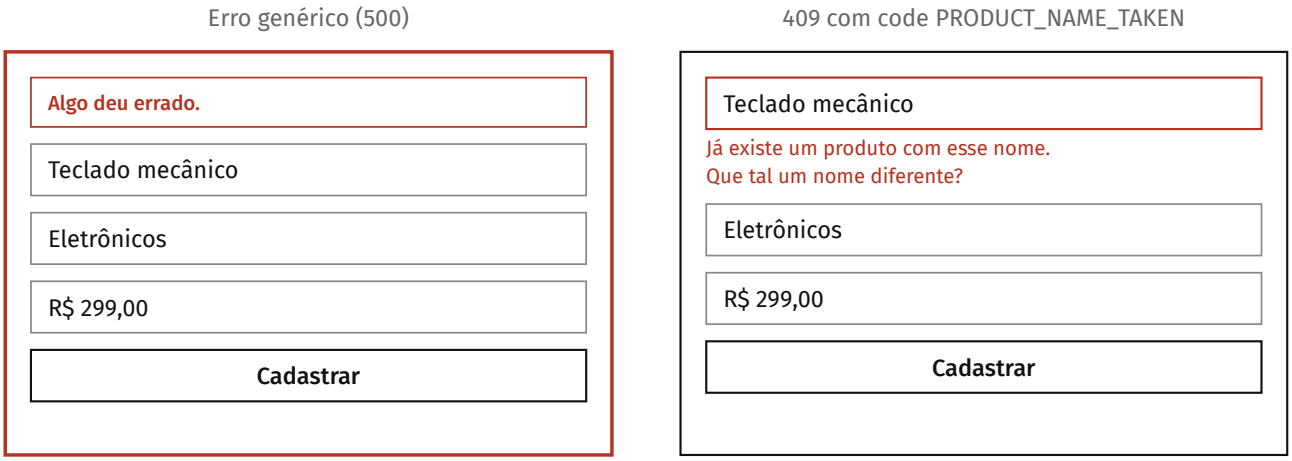


Figura 2.1. O mesmo envio nas duas versões. Com o code por campo, a mensagem aparece onde está o problema.

2.2 Por que acontece

O status code diz de quem é o problema. A família 4xx quer dizer que a requisição tem algo errado e que repetir igual não vai adiantar; a família 5xx quer dizer que o servidor falhou e que talvez dê certo mais tarde. Um `500` para um nome repetido diz o contrário da verdade: que o servidor quebrou, quando quem precisa mudar algo é a pessoa.

Os mais comuns no dia a dia: `400` para uma requisição mal formada, `401` para quem não tem sessão (tema 16, Sessão expirada), `403` para quem não tem permissão, `404` para o que não existe, `409` para conflito

com o estado atual, `422` para dados que não passam na validação e `429` para excesso de requisições (tema 15, Rate limiting).

Só o status, porém, não basta. Um `422` diz que algum dado está errado, mas não qual nem por quê. Se o detalhe vier só numa frase, o front precisa procurar palavras no texto para saber o que aconteceu, e qualquer ajuste na frase quebra a tela.

2.3 Na API: status certo e um code para cada erro

Toda resposta de erro tem o mesmo formato: uma lista `errors`, em que cada item traz um `code`, uma `message` e, quando o erro é de um campo, o `field`. O *code* é um nome fixo em maiúsculas, como `PRODUCT_NAME_TAKEN`, que não muda quando alguém reescreve a frase. A validação vem primeiro e responde `422` com todos os campos errados de uma vez; só com os dados certos a API consulta o banco, e o nome repetido vira `409`.

API · Servidor

```
app.post('/api/novos-produtos', async (req, res) => {
  const errors = validar(req.body)              // [{ code, message, field }] ou []
  if (errors.length > 0) {
    return res.status(422).json({ errors })      // os dados não passam
  }

  if (await nomeEmUso(req.body.nome)) {          // dados certos, mas o nome já existe
    return res.status(409).json({ errors: [{
      code: 'PRODUCT_NAME_TAKEN',
      message: 'Já existe um produto com esse nome',
      field: 'nome',
    }] })
  }

  res.status(201).json(await salvar(req.body))
})
```

2.4 No front: o texto sai do code

O front não lê a frase da API para decidir nada: ele compara o code. A mensagem que aparece na tela vem de uma tabela do próprio front, de code para texto, que pode virar a chave de tradução quando a aplicação tiver outros idiomas. Na lojinha, a frase do `PRODUCT_NAME_TAKEN` no front é diferente da que a API manda, de propósito, para mostrar quem decide o texto. Se chegar um code que o front ainda não conhece, ele usa a `message` da API.

Frontend · Navegador

```
const MENSAGENS = {
  PRODUCT_NAME_REQUIRED: 'Informe o nome do produto',
  PRODUCT_NAME_TAKEN: 'Já existe um produto com esse nome. Que tal um nome diferente?',
  PRODUCT_CATEGORY_INVALID: 'Escolha uma das categorias',
  PRODUCT_PRICE_INVALID: 'Informe um preço maior que zero',
}

const resposta = await fetch('/api/novos-produtos', { method: 'POST', body, headers })
if (resposta.status === 422 || resposta.status === 409) {
  const { errors } = await resposta.json()
  for (const { code, message, field } of errors) {
    mostrarErroNoCampo(field, MENSAGENS[code] ?? message)   // code novo: usa a da API
  }
} else if (!resposta.ok) {
  mostrarAviso('Não foi possível cadastrar agora. Seus dados continuam aqui.')
}
```

2.5 Cuidados

O code é um contrato. Depois que o front depende de `PRODUCT_NAME_TAKEN`, renomear o code quebra a tela, como renomear um campo da API. Pelo mesmo motivo, um code genérico para tudo, como `VALIDATION_ERROR`, devolve o front ao problema de ler a frase.

A mensagem é para gente. Ela não deve trazer stack trace, nome de tabela ou SQL, que só ajudam quem quer atacar a API. E o formato precisa ser o mesmo em todas as rotas, para o front tratar erro num lugar só; se quiser um formato pronto, existe um padrão para isso, o Problem Details (RFC 9457).

Separar 4xx de 5xx também decide o que fazer depois. Um `422` não melhora repetindo; um `503` talvez melhore em alguns segundos, e o front pode tentar de novo sozinho (tema 13, Retries e exponential backoff).

2.6 No seu projeto

Mande para a sua API, por curl ou Insomnia, um cadastro com um campo inválido. Qual status volta? Dá para saber, só pela resposta e sem ler a frase, qual campo corrigir e por quê?

Tema 3 · Frontend

URL como estado

Filtros, busca, página atual e aba escolhida são estado da tela. Quando esse estado fica só na memória do componente, ele some no F5 e não vai junto quando alguém copia o link. Guardar esse estado na URL resolve as duas coisas.

3.1 O problema

Na lojinha, a pessoa busca “garrafa” e escolhe a categoria Casa. Acha a garrafa térmica e manda o link para um amigo, que abre a loja na primeira categoria, sem busca nenhuma. Ela mesma dá F5 para atualizar o preço e perde o filtro. O botão Voltar do navegador também não volta para o que ela estava vendo.

Na lojinha, o tema tem duas versões: Filtros no useState e Filtros na URL.

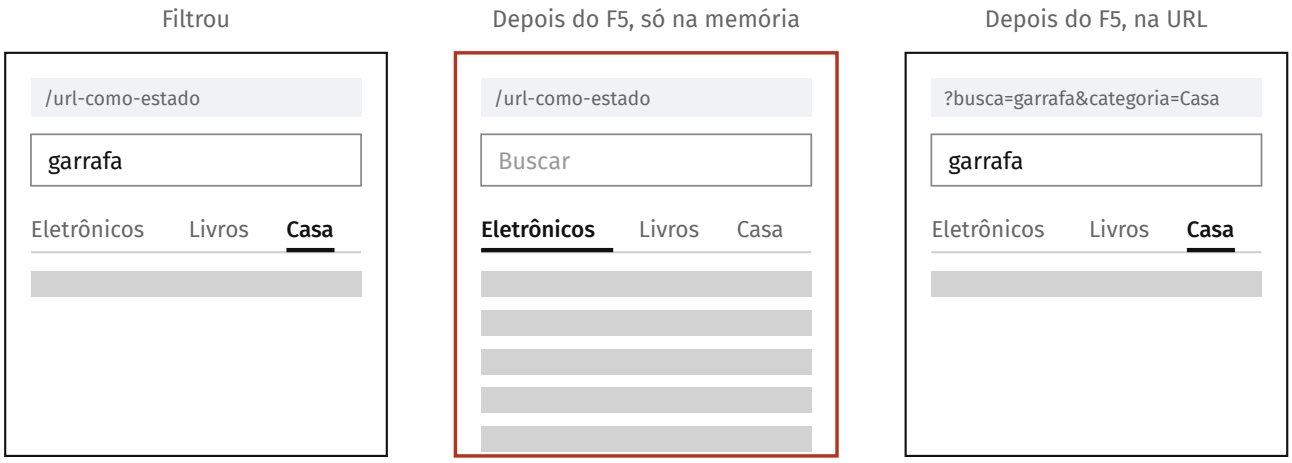


Figura 3.1. Com os filtros só na memória, o F5 volta ao começo. Com os filtros na URL, eles voltam junto.

3.2 Por que acontece

Um `useState`, ou qualquer variável em JavaScript, vive na memória daquela aba. O F5 recria a página do zero e essa memória vai embora. O link copiado só carrega o que está escrito na URL, e se os filtros não estão lá, quem abre o link vê a página inicial.

3.3 No front: ler e escrever os filtros na URL

A tela lê os filtros da URL ao abrir e, a cada mudança, escreve de volta, sem recarregar a página. Em React com React Router, o `useSearchParams` faz as duas coisas.

Frontend · Navegador

```
// Ao abrir (e depois de um F5): os filtros vêm da URL
const parametros = new URLSearchParams(location.search)
const busca = parametros.get('busca') ?? ''
const categoria = parametros.get('categoria') ?? 'Eletrônicos'

// Ao mudar um filtro: atualiza a URL sem recarregar
function mudarFiltro(nome, valor) {
  const url = new URL(location.href)
  if (valor) url.searchParams.set(nome, valor)
  else url.searchParams.delete(nome)
  history.replaceState(null, '', url)    // ?busca=garrafa&categoria=Casa
}
```

3.4 Cuidados

A diferença entre `replaceState` e `pushState` aparece no Voltar. Com `push` a cada tecla, o Voltar desfaz a busca letra por letra; para campos de texto, use `replace`. Para escolhas feitas com clique, como a página da lista, `push` pode ser o que a pessoa espera.

Ponha na URL só o que faz sentido compartilhar. Ela vai para o histórico, para logs de servidor e para quem recebe o link; token e dado pessoal não entram. E trate o que vem da URL como entrada de usuário: qualquer um pode editar. Um link compartilhado também pode abrir uma busca que não encontra nada, e a tela precisa dizer isso direito (tema 4, Estados da tela).

3.5 No seu projeto

Filtre uma lista, dê F5 e depois cole o link numa aba anônima. A tela volta igual nos dois casos?

Bloco 2

Durante a espera

Entre o clique e a resposta existe um intervalo que pode durar 50 milissegundos ou 30 segundos. O que a tela mostra nesse intervalo decide se a espera parece curta, se a pessoa clica de novo ou se ela acaba vendo o resultado errado.

4	Estados da tela	Frontend
5	Skeleton loading	Frontend
6	Layout shift	Frontend
7	Debounce e throttle	Frontend
8	Race conditions e AbortController	Frontend
9	Optimistic UI	Frontend + API
10	Feedback de sucesso	Frontend
11	Upload com progresso e tempo real	Frontend + API

Tema 4 · Frontend

Estados da tela

Toda tela que busca dados passa por quatro estados: carregando, erro, vazio e sucesso. Em geral só o último é desenhado com cuidado; os outros três ficam como uma área em branco ou um “nada encontrado” solto.

4.1 O problema

Na versão ruim da lojinha, a lista de produtos mostra a mesma coisa em três situações diferentes: enquanto carrega, quando a API falha e quando a busca não encontra nada. Nas três, uma área em branco. A pessoa não sabe se espera, se tenta de novo ou se muda a busca, e quase sempre escolhe a pior opção: recarregar a página.

Na lojinha, o tema tem duas versões: Tela branca e Quatro estados. Para ver o erro, use a taxa de erro em 100% no monitor; para o vazio, busque “xyz”.

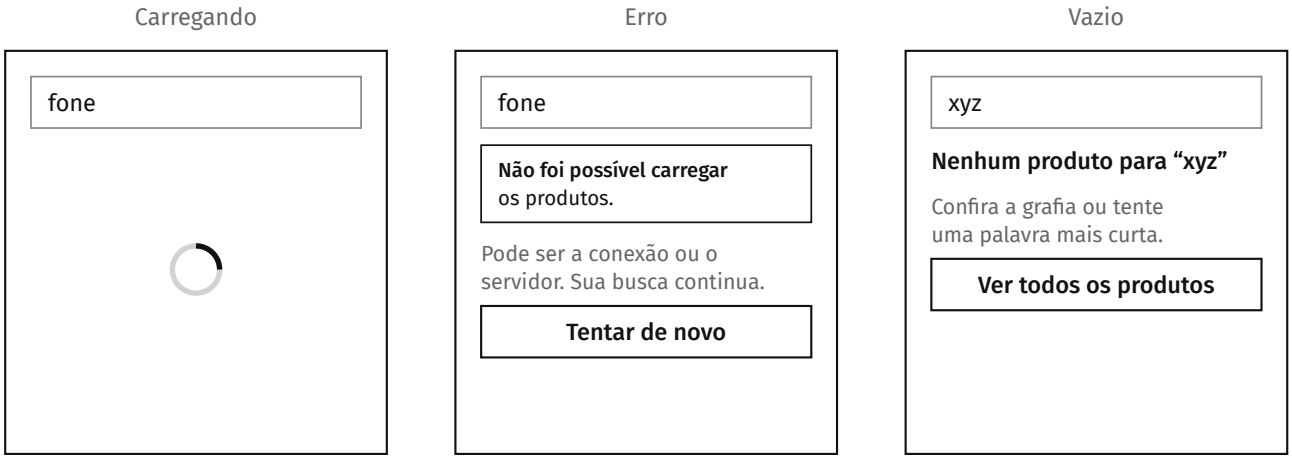


Figura 4.1. Os três estados que costumam ser esquecidos, cada um com o seu desenho. O quarto é a lista.

4.2 Por que acontece

O código costuma nascer pensando no caminho feliz. Com `const [produtos, setProdutos] = useState([])`, a lista vazia do começo é igual à lista vazia de uma busca sem resultado, e se a promise falha sem `catch`, o erro simplesmente some. A tela não tem como desenhar estados que o código não distingue.

4.3 No front: um desenho para cada estado

O carregando mostra que algo está acontecendo. O erro diz o que houve, em palavras de gente, e oferece “Tentar de novo”. O vazio explica por que não há nada e sugere o próximo passo, como conferir a grafia ou ver

todos os produtos. Só depois vem a lista.

Frontend · Navegador

```
if (carregando) return mostrarCarregando()

if (erro) return mostrarErro('Não foi possível carregar os produtos.', tentarDeNovo)

if (produtos.length === 0) {
  return mostrarVazio(`Nenhum produto encontrado para “${busca}”`, verTodos)
}

return mostrarLista(produtos)
```

Bibliotecas como o TanStack Query já entregam esses estados separados, como `isPending`, `isError` e `refetch`, e evitam boa parte dos esquecimentos.

4.4 Cuidados

A ordem das perguntas importa: o erro vem antes do vazio, porque uma lista vazia por falha não é “nenhum produto encontrado”. E o vazio de uma busca é diferente do vazio de uma conta nova; o primeiro sugere mudar a busca, o segundo sugere criar o primeiro item.

O estado que a pessoa mais vê é o de carregando, e o jeito de desenhá-lo muda a sensação de quanto tempo passou (tema 5, Skeleton loading).

4.5 No seu projeto

Em cada tela que carrega dados, force os três casos: DevTools em Offline para o erro, uma busca absurda para o vazio e a rede em Slow 4G para o carregando. Cada um tem um desenho próprio?

Tema 5 · Frontend

Skeleton loading

Skeleton é o esqueleto da tela desenhado antes de os dados chegarem: blocos cinza no formato do que vai aparecer. O tempo de espera é o mesmo de um spinner, mas a sensação é de que a página já está quase pronta.

5.1 O problema

Na lojinha, trocar de categoria leva um segundo. Na versão com spinner, a lista some, um círculo gira no meio de uma área vazia e, de repente, tudo aparece de uma vez, empurrando o que estava em volta. A espera parece mais longa do que é, e o salto no fim faz a pessoa perder o ponto onde estava olhando.

Na lojinha, o tema tem duas versões: Spinner e Skeleton. Use a latência de 1 s no monitor.



Figura 5.1. A mesma espera de um segundo. O skeleton já tem o formato da lista que vai chegar.

5.2 Por que acontece

Entre um e dez segundos, a pessoa percebe a espera e procura sinais de que algo está andando. O spinner diz só “espere”, sem dizer o quê nem quanto. O skeleton mostra a estrutura: o olho já encontra onde vão ficar os nomes e os preços, e quando os dados chegam, eles ocupam um lugar que já existia.

5.3 No front: desenhar o skeleton no formato real

O skeleton usa a mesma grade e os mesmos cards da lista, com blocos cinza no lugar de cada texto. Uma animação leve, pulsando, mostra que a página não travou.

Frontend · CSS

```css
.osso {
  background: #e9ecef;
  border-radius: 4px;
  animation: pulsar 1.5s ease-in-out infinite;
}

@keyframes pulsar { 50% { opacity: .5 } }
```

Frontend · HTML

```html
<div class="card">
  <div class="osso" style="width: 70%; height: 16px"></div>   <!-- nome -->
  <div class="osso" style="width: 100px; height: 22px"></div>  <!-- preço -->
</div>
```

5.4 Cuidados

O skeleton serve para esperas curtas de conteúdo cujo formato você conhece. Para uma ação, como salvar, um botão “Salvando…” diz mais. Para esperas longas, a pessoa precisa de progresso de verdade (tema 11, Upload com progresso e tempo real). E se a resposta costuma chegar em menos de 300 ms, o skeleton aparece e some tão rápido que vira um piscar; vale atrasar sua entrada.

Um skeleton de tamanho diferente da tela real vira, ele mesmo, um salto quando os dados chegam. Esse salto tem nome e medida própria (tema 6, Layout shift).

5.5 No seu projeto

Com a rede em Slow 4G, abra a tela principal. Durante a espera, dá para saber onde vai aparecer cada coisa? Quando os dados chegam, algo muda de lugar?

Tema 6 · Frontend

Layout shift

Layout shift é quando um elemento muda de lugar depois que a página já apareceu, porque algo carregou acima ou ao lado dele. O Google mede isso numa das Core Web Vitals, o CLS (Cumulative Layout Shift), que também pesa na posição do site na busca.

6.1 O problema

A lojinha mostra uma grade de produtos com foto, nome e botão Comprar. As fotos não têm tamanho declarado e chegam uma a uma, em momentos diferentes. A pessoa mira no Comprar do Mouse gamer, a foto termina de carregar bem nessa hora, tudo desce, e o clique cai na foto, ou no botão de outro produto.

Na lojinha, o tema tem duas versões: Imagem sem tamanho e Espaço reservado. Use a latência Aleatória no monitor.



Figura 6.1. O cursor fica parado. Sem espaço reservado, a foto chega e empurra o botão para baixo do cursor.

6.2 Por que acontece

O navegador desenha a página antes de saber o tamanho de tudo. Uma `<img>` sem `width` e `height` começa com altura zero; quando o arquivo chega, o navegador descobre o tamanho e redesenha, empurrando o que vem depois. O mesmo acontece com banners, anúncios, avisos inseridos no topo e fontes que trocam de tamanho ao carregar.

6.3 No front: reservar o espaço

Basta dizer a proporção antes de a imagem chegar. Com `width` e `height` no HTML, o navegador calcula a altura pela largura disponível e guarda o lugar; o CSS ainda pode deixar a imagem ocupar a largura toda.

Frontend · HTML

```html
<!-- width e height dão a proporção (4:3) antes de a imagem chegar -->
<img src="/api/imagens/5" alt="Mouse gamer"
     width="640" height="480"
     style="width: 100%; height: auto">
```

O mesmo efeito sai em CSS puro com `aspect-ratio: 4 / 3`. Para avisos e banners que chegam depois, o caminho é reservar a altura deles desde o começo ou mostrá-los por cima do conteúdo, sem empurrar nada.

6.4 Cuidados

Mudança causada pelo próprio clique da pessoa, como abrir um menu, não conta como problema; o CLS ignora o que acontece logo depois de uma interação. O problema é o que se mexe sem a pessoa pedir.

Reservar o espaço é obrigatório no lazy loading. Sem altura, o navegador acha que todas as imagens cabem na tela e baixa todas de uma vez (tema 20, Lazy loading).

6.5 No seu projeto

No DevTools, abra Rendering e ligue Layout Shift Regions: o que se mover fica pintado de azul. Recarregue a página com a rede em Slow 4G e observe. O Lighthouse também mostra o valor do CLS.

Tema 7 · Frontend

Debounce e throttle

Os dois limitam quantas vezes uma função roda quando um evento dispara em rajada. O debounce espera a rajada acabar e roda uma vez só, no fim. O throttle roda no máximo uma vez a cada intervalo, durante a rajada.

7.1 O problema

A busca da lojinha filtra enquanto a pessoa digita. Sem debounce, cada tecla dispara uma requisição: “fone de ouvido” são 14 teclas e 14 requisições, das quais só a última interessa. A lista pisca a cada letra, a API trabalha catorze vezes e, no celular, o plano de dados paga.

Na lojinha, o tema tem três versões: Sem debounce, Com debounce e Throttle no scroll, com um contador de requisições na tela.

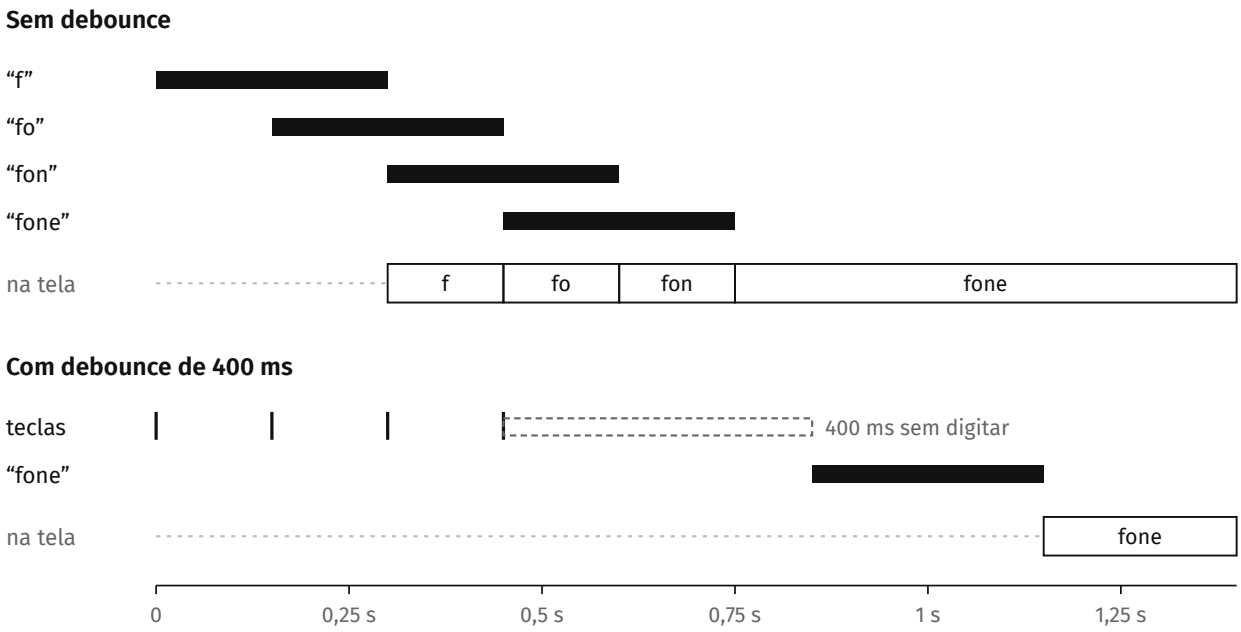


Figura 7.1. As mesmas quatro teclas. Com debounce, a busca só sai depois de 400 ms sem digitar.

7.2 Por que acontece

O evento `input` dispara a cada tecla, e `scroll` e `resize` disparam dezenas de vezes por segundo. Código que reage direto a cada evento faz trabalho que vai ser jogado fora logo em seguida: a busca de “fon” não interessa a quem está digitando “fone”.

7.3 No front: debounce na busca, throttle no scroll

O debounce reinicia um timer a cada tecla; a função só roda quando o timer chega ao fim sem ser interrompido. Entre 300 e 500 ms é o intervalo usual para digitação.

Frontend · Navegador

```
function debounce(funcao, atraso) {
  let timer
  return (...args) => {
    clearTimeout(timer)                          // chegou outra tecla: recomeça a contar
    timer = setTimeout(() => funcao(...args), atraso)
  }
}

campo.addEventListener('input', debounce((e) => buscar(e.target.value), 400))
```

O throttle serve quando a tela precisa reagir durante o movimento, e não só no fim, como uma barra que acompanha o scroll. Ele deixa passar uma chamada e ignora as seguintes até o intervalo acabar.

Frontend · Navegador

```
function throttle(funcao, intervalo) {
  let ultimaVez = 0
  return (...args) => {
    if (Date.now() - ultimaVez < intervalo) return  // ainda dentro do intervalo: ignora
    ultimaVez = Date.now()
    funcao(...args)
  }
}

window.addEventListener('scroll', throttle(atualizarPosicao, 200))
```

7.4 Cuidados

Debounce alto demais faz a busca parecer lenta, porque nada acontece até a pessoa parar. E o throttle simples acima pode perder o último evento, deixando a tela um passo atrás; a versão da lojinha agenda a última chamada para o fim do intervalo.

O debounce diminui quantas requisições saem, mas não garante a ordem em que as respostas voltam. E em cliques ele não serve: um clique é uma escolha, e atrasá-lo só deixa a tela lenta. Quando a resposta antiga chega depois da nova, o problema é outro (tema 8, Race conditions e AbortController).

7.5 No seu projeto

Com a aba Network aberta, digite uma palavra na busca do seu sistema.
Quantas requisições saíram?

Tema 8 · Frontend

Race conditions e AbortController

Uma *race condition* acontece quando duas operações correm ao mesmo tempo e o resultado depende de qual termina primeiro. No front, o caso típico é a tela pedir uma coisa nova à API antes de a anterior responder: as respostas chegam fora de ordem e a mais lenta apaga a mais recente. O `AbortController` é o recurso do navegador para cancelar a requisição que deixou de interessar.

8.1 O problema

A lojinha separa os produtos em abas de categoria. A pessoa clica em Eletrônicos, muda de ideia e clica em Casa logo em seguida. Os produtos de Casa aparecem quase na hora, e tudo parece certo. Ela começa a olhar a lista e, dois segundos depois, a lista troca sozinha: a aba marcada continua sendo Casa, mas a tela mostra fones, teclados e monitores.

Ninguém clicou em nada errado. Quem não percebe a troca pode pôr no carrinho algo achando que é de outra categoria; quem percebe passa a desconfiar da loja e recarrega a página para garantir.

Na lojinha, o tema tem duas versões: Sem cancelamento e Com AbortController. Eletrônicos sempre demora 2 s; as outras categorias, 0,3 s.



Figura 8.1. A mesma tela em três momentos. Aos 0,4 s está certa; aos 2 s, a aba diz Casa e a lista é de Eletrônicos.

8.2 Por que acontece

Cada clique dispara um `fetch`, e os `fetch` não sabem uns dos outros. O navegador não tem como saber que o clique em Casa tornou a resposta de Eletrônicos inútil, e a rede não promete ordem. Uma categoria com mais produtos, uma consulta mais pesada ou um servidor mais ocupado

Ver também o tema 7, Debounce e throttle, que resolve outro problema: requisições demais enquanto se digita.

bastam para a primeira resposta chegar depois da segunda. Se o código guarda na tela qualquer resposta que chega, a última a chegar ganha, e ela nem sempre é a do último clique.

A figura 8.2 mostra os dois cliques no tempo. Cada barra vai do clique até a resposta; a faixa de baixo é o que a tela está mostrando.

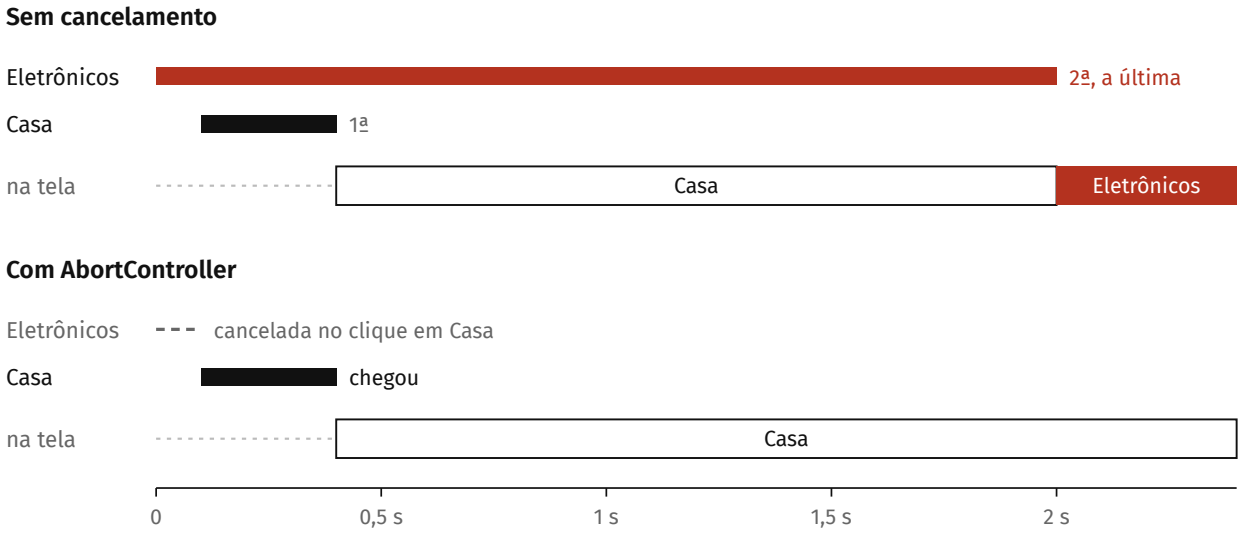


Figura 8.2. Os dois cliques no tempo, sem e com cancelamento.

Debounce não ajuda aqui. Ele serve para quem digita, em que as letras do meio não interessam. Um clique numa aba é uma escolha, e atrasar cada clique só deixaria a tela mais lenta.

8.3 No front: cancelar a requisição anterior

A saída é tratar a requisição antiga como descartável assim que chega um clique novo. O `fetch` recebe o `signal` de um `AbortController`; quando o código chama `abort()`, o navegador fecha a requisição e a promise falha com um erro chamado `AbortError`. A resposta velha nunca chega ao código que atualiza a tela.

Frontend · Navegador

```
let controle

async function escolherCategoria(categoria) {
  marcarAba(categoria)
  controle?.abort()                 // cancela a requisição do clique anterior
  controle = new AbortController()

  try {
    const resposta = await fetch(`/api/produtos?categoria=${categoria}`, {
      signal: controle.signal,
    })
    mostrarProdutos(await resposta.json())
  } catch (erro) {
    if (erro.name !== 'AbortError') throw erro  // cancelamos de propósito
  }
}
```

O `catch` precisa separar os dois casos. O cancelamento foi pedido pelo próprio código e não é falha; qualquer outro erro continua sendo erro e deve aparecer para o usuário. Em React, o controller do clique anterior precisa sobreviver entre um render e outro, então ele fica num `useRef`; o resto é igual.

Quando cancelar não é possível, por exemplo com uma biblioteca que não aceita `signal`, dá para chegar ao mesmo resultado ignorando a resposta: guarde a categoria junto com os produtos e só mostre se ela ainda for a aba marcada.

8.4 Cuidados

Cancelar no navegador não para o servidor. O `abort()` fecha a conexão do lado do cliente; a API já recebeu a requisição, consulta o banco do mesmo jeito e responde `200` para ninguém. Numa leitura simples, tudo bem. Se a consulta for cara, a API pode perceber que a conexão fechou e interromper o trabalho, mas isso é decisão do servidor.

Na aula, o monitor mostrou a barra de Eletrônicos cancelada no navegador enquanto a API seguia até o fim e respondia 200.

Pelo mesmo motivo, cancelar só serve para leituras. Abortar um `POST` de compra não desfaz a compra: o pedido pode já ter sido gravado. Para ações que mudam dados, o caminho é outro (tema 14, Clique duplo e idempotência).

8.5 No seu projeto

Abra o DevTools, coloque a rede em Slow 4G e clique rápido entre abas, filtros ou páginas de uma lista. Se a tela nem sempre termina mostrando o que foi clicado por último, falta cancelar ou descartar as respostas antigas.

Tema 9 · Frontend + API

Optimistic UI

Optimistic UI é mudar a tela antes de a API responder, apostando que ela vai aceitar. Se a resposta vier com erro, a tela volta atrás e avisa. Serve para ações pequenas que quase sempre dão certo, como curtir, favoritar ou marcar como lida.

9.1 O problema

Na lojinha, cada produto tem um coração para favoritar. Na versão que espera a API, com a latência de um segundo, a pessoa clica e nada acontece. Um segundo parado depois de um clique parece defeito, então ela clica de novo, e o segundo clique desfaz o primeiro.

Na lojinha, o tema tem duas versões: Espera a API e Muda na hora. Use a latência de 1 s e depois a taxa de erro em 100%.



Figura 9.1. Esperando a API, o coração demora; mudando na hora, ele responde ao clique e só volta se a API recusar.

9.2 Por que acontece

Quando a tela só muda depois da resposta, o tempo do clique passa a depender da rede. Até 0,1 segundo a pessoa sente que causou o efeito; um segundo depois, ela já não liga uma coisa à outra. Numa ação que dá certo em quase todos os casos, esperar a confirmação é pagar sempre por um erro que quase nunca acontece.

9.3 No front: mudar antes, desfazer se falhar

A tela muda no clique. A requisição sai em seguida, e se ela falhar, a tela volta ao estado anterior e diz o que aconteceu. Voltar calado seria pior do que esperar.

Frontend · Navegador

```
async function alternarFavorito(produto) {
  const favoritar = !favoritos.has(produto.id)
  marcar(produto.id, favoritar)                          // 1. muda na hora

  try {
    const resposta = await fetch(`/api/favoritos/${produto.id}`, {
      method: favoritar ? 'PUT' : 'DELETE',
    })
    if (!resposta.ok) throw new Error(resposta.status)
  } catch {
    marcar(produto.id, !favoritar)                       // 2. a API recusou: desfaz e avisa
    avisar(`Não foi possível favoritar “${produto.nome}”. Tente de novo.`)
  }
}
```

9.4 Na API: uma operação que pode chegar duas vezes

Com a tela respondendo na hora, a API precisa aguentar cliques rápidos e repetidos. Por isso favoritar é `PUT /favoritos/7` e desfavoritar é `DELETE /favoritos/7`, e não um `POST /favoritos/7/alternar`. Com “alternar”, duas requisições iguais voltam ao começo; com `PUT` e `DELETE`, repetir dá no mesmo, porque os dois são idempotentes. No banco, o `PUT` vira um `INSERT ... ON CONFLICT DO NOTHING`, e o `DELETE` apaga se existir.

9.5 Cuidados

Optimistic UI é para ações reversíveis e com alta chance de sucesso. Pagamento, envio de mensagem importante ou qualquer coisa que dependa de uma validação do servidor ficam de fora: mostrar “pago” e depois voltar atrás é pior do que esperar dois segundos.

Quando a ação não aparece na própria tela, como adicionar algo a um carrinho que está em outra página, mudar o coração não basta, e a pessoa precisa de outra confirmação (tema 10, Feedback de sucesso).

9.6 No seu projeto

Com a rede em Slow 4G, clique em curtir ou favoritar. A tela responde antes de a requisição terminar? Agora coloque o DevTools em Offline e clique de novo: a tela volta e avisa?

Tema 10 · Frontend

Feedback de sucesso

Depois de uma ação, a pessoa precisa de um sinal de que deu certo. Esse sinal tem que ser visível sem atrapalhar: nem silêncio, nem um alerta que trava a tela só para dizer “Sucesso!”.

10.1 O problema

Na lojinha, o botão Adicionar ao carrinho tem três versões. Na primeira, o produto vai para o carrinho, mas a tela não muda; a pessoa clica de novo e fica com dois. Na segunda, abre uma janela “Produto adicionado!” com um botão OK: o sinal existe, mas cada produto custa um clique a mais e tira a pessoa do que ela estava fazendo. Na terceira, uma confirmação aparece num canto e some sozinha.

Na lojinha, o tema tem três versões: Nada acontece, Alerta que bloqueia e Toast.

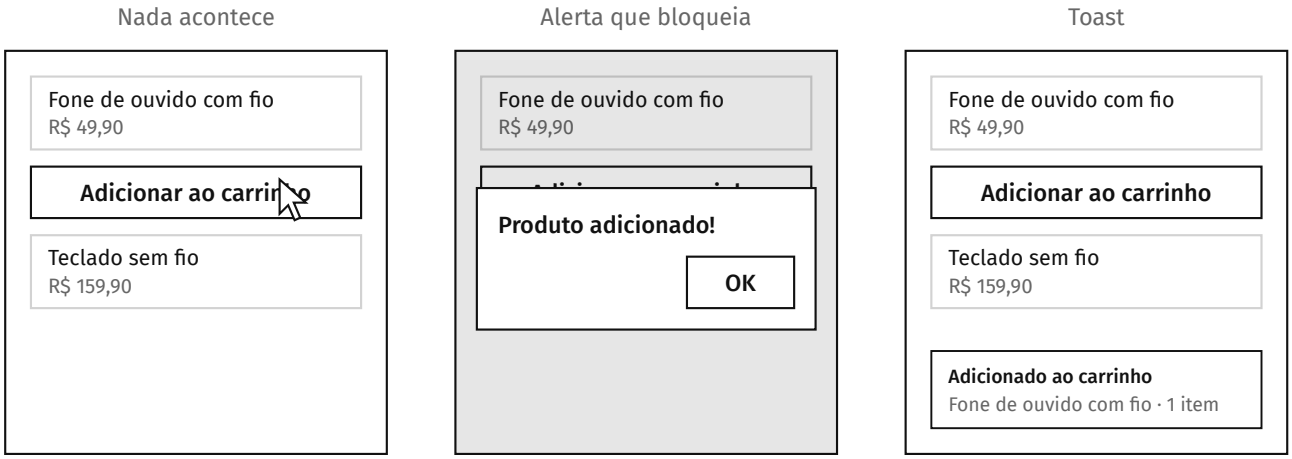


Figura 10.1. A mesma ação nas três versões. O toast confirma sem pedir nada à pessoa.

10.2 Por que acontece

Algumas ações não mudam nada na tela que a pessoa está vendo: o carrinho está em outra página, o e-mail foi para outra pessoa, o arquivo foi salvo num servidor. Sem sinal, ela repete a ação ou acha que falhou. O alerta que bloqueia resolve o sinal, mas exige um clique para continuar, e com o tempo a pessoa aprende a fechá-lo sem ler.

10.3 No front: confirmação discreta

O toast aparece num canto, diz o que aconteceu com o nome do item, some sozinho em uns três segundos e não impede nada. Se existir um contador do carrinho no cabeçalho, ele também muda na hora. Na

lojinha, o toast vem do Mantine.

Frontend · Navegador

```
import { notifications } from '@mantine/notifications'

async function adicionar(produto) {
  const { itens } = await adicionarAoCarrinho(produto)

  notifications.show({
    title: 'Adicionado ao carrinho',
    message: `${produto.nome} · ${itens} itens no carrinho`,
    autoClose: 3000,                    // some sozinho, sem pedir clique
  })
}
```

10.4 Cuidados

Toast é para confirmar, não para erro que exige ação: um erro que precisa ser resolvido fica na tela até ser resolvido. Leitores de tela só anunciam o toast se ele estiver numa região com `role="status"` ou `aria-live`; bibliotecas como o Mantine já fazem isso. E quando dá para mostrar o resultado na própria tela, como o botão que vira “No carrinho”, isso costuma ser melhor que qualquer aviso.

Para ações que demoram, um “pronto” no fim não basta: a pessoa precisa acompanhar o caminho (tema 11, Upload com progresso e tempo real).

10.5 No seu projeto

Faça cada ação importante do seu sistema, como salvar, adicionar e enviar, olhando só para a tela. Dá para saber que funcionou sem abrir outra página?

Tema 11 · Frontend + API

Upload com progresso e tempo real

Subir um arquivo tem duas esperas: o envio dos bytes, que o navegador consegue medir, e o processamento no servidor, que só a API conhece. Para mostrar o progresso da segunda, o servidor precisa contar novidades ao front, e há três jeitos de fazer isso: short polling, long polling e SSE.

11.1 O problema

Na lojinha, a pessoa sobe uma foto de 10 MB. Na versão sem progresso, o botão gira enquanto o arquivo vai e depois a tela fica parada em "Processando..." por oito segundos, enquanto o servidor verifica o arquivo e gera as miniaturas. Ela não sabe se travou. Fecha a aba ou manda o arquivo de novo.

Na lojinha, o tema tem quatro versões: Sem progresso, Short polling, Long polling e SSE. Use um arquivo de uns 10 MB.

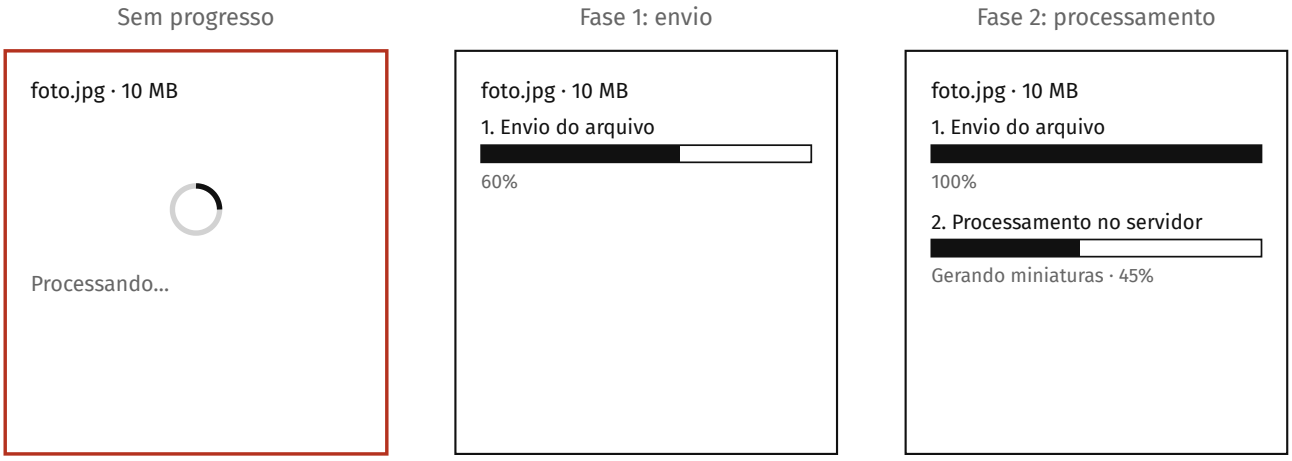


Figura 11.1. Sem progresso, a tela não diz nada depois do envio. Com progresso, cada fase tem a sua barra.

11.2 Por que acontece

Na primeira fase, quem sabe quantos bytes já foram é o navegador, e ele avisa pelo evento `xhr.upload.onprogress`; o `fetch` não expõe o progresso de envio de forma simples. Na segunda fase, só o servidor sabe em que ponto está, e no HTTP comum quem pergunta é sempre o cliente. A figura 11.2 compara os três jeitos de o front ficar sabendo.

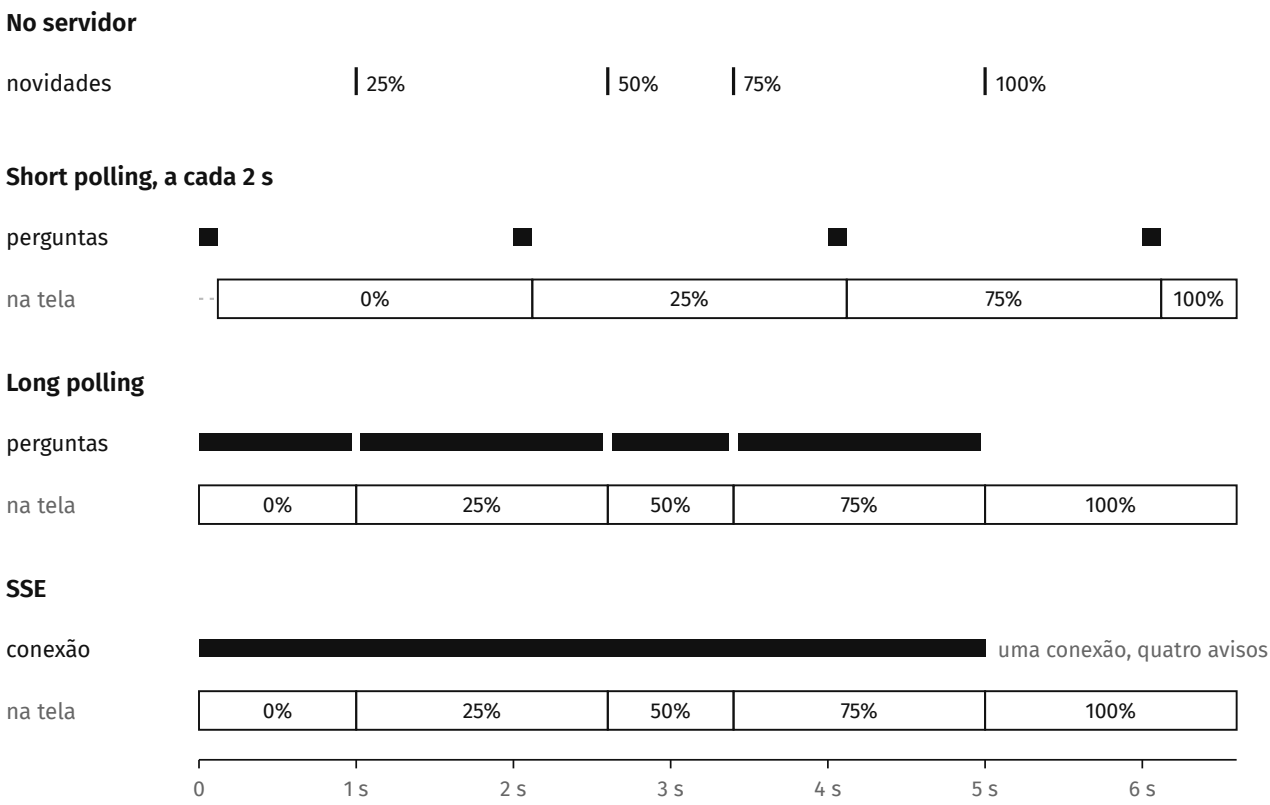


Figura 11.2. As mesmas quatro novidades no servidor. O short polling chega atrasado e até pula a de 50%; o long polling e o SSE mostram cada uma na hora.

No short polling, o front pergunta de tempos em tempos e a API responde na hora, tenha novidade ou não. No long polling, a API segura a resposta até ter novidade, e o front pergunta de novo assim que ela chega. No SSE (Server-Sent Events), o front abre uma conexão só e a API escreve nela cada mudança. É o mesmo mecanismo que o ChatGPT e o Claude usam para mostrar a resposta palavra por palavra.

11.3 No front: medir o envio e escutar o processamento

Frontend · Navegador

```
const xhr = new XMLHttpRequest()
xhr.upload.onprogress = (e) => mostrarEnvio(Math.round((e.loaded / e.total) * 100))
xhr.onload = () => acompanhar(JSON.parse(xhr.responseText).id)
xhr.open('POST', '/api/uploads')
xhr.send(arquivo)

function acompanhar(id) {
  const fonte = new EventSource(`/api/uploads/${id}/eventos`)
  fonte.onmessage = (mensagem) => {
    const { progresso, etapa } = JSON.parse(mensagem.data)
    mostrarProcessamento(progresso, etapa)        // "Gerando miniaturas · 45%"
    if (progresso === 100) fonte.close()          // senão o EventSource reconecta
  }
}
```

11.4 Na API: responder logo e avisar depois

O `POST` do upload grava o arquivo, responde `201` com o id e deixa o processamento rodando à parte; a pessoa não fica presa numa requisição de oito segundos. Quem processa grava o progresso no banco, e o Postgres avisa a API com `NOTIFY`. A API escuta com `LISTEN` e repassa cada aviso pela conexão SSE.

API · Servidor

```
app.get('/api/uploads/:id/eventos', (req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/event-stream', 'Cache-Control': 'no-cache' })

  const enviar = (status) => {
    if (status.id !== Number(req.params.id)) return
    res.write(`data: ${JSON.stringify(status)}\n\n`)
  }
  avisos.on('progresso', enviar)                  // avisos chegam do LISTEN do Postgres
  req.on('close', () => avisos.off('progresso', enviar))
})
```

Situação	Use
Novidade rara, atraso tolerável, checagem curta (o status de um Pix por um minuto)	Short polling
Precisa ser na hora, mas a infraestrutura não aceita conexão aberta	Long polling
O servidor fala e o cliente só escuta (progresso, notificações, texto sendo gerado)	SSE
Os dois lados falam o tempo todo (chat, jogo, edição colaborativa)	WebSocket

11.5 Cuidados

Polling multiplica com o número de pessoas: mil usuários perguntando a cada dois segundos são 500 requisições por segundo, quase todas respondendo “nada mudou”. O SSE troca isso por uma conexão aberta por pessoa, e alguns proxies e plataformas serverless derrubam conexões longas. O `EventSource` também não aceita headers próprios, então a autenticação vai por cookie.

Uma barra parada pode ser um servidor processando ou um servidor travado. Para o segundo caso, o front precisa de um limite de espera (tema 12, Timeouts).

11.6 No seu projeto

Suba um arquivo grande com a rede em Slow 4G. Dá para ver quanto falta no envio? E depois que o envio termina, a tela diz o que está acontecendo?

Bloco 3

Quando dá errado

A rede cai, o servidor trava, a sessão expira, alguém clica duas vezes. Nenhuma dessas falhas é rara, e cada uma tem um jeito de não virar um “Algo deu errado” que apaga o trabalho da pessoa.

12	Timeouts	Frontend + API
13	Retries e exponential backoff	Frontend + API
14	Clique duplo e idempotência	Frontend + API
15	Rate limiting	API
16	Sessão expirada	Frontend + API
17	Offline e conexão instável	Frontend + API

Tema 12 · Frontend + API

Timeouts

Timeout é o tempo máximo que se aceita esperar por uma resposta. O `fetch` não tem um por padrão: se a API travar, ele espera para sempre, e o spinner gira junto.

12.1 O problema

Na lojinha, com a API travada, a lista de produtos fica no spinner. Dez segundos, vinte, trinta. A pessoa não sabe se é lentidão normal, se a internet caiu ou se a loja quebrou, e não há botão para tentar de novo. Ela recarrega a página, que trava do mesmo jeito, ou desiste da compra.

Na lojinha, o tema tem duas versões: Sem limite e Com timeout. Use a latência Travada (30 s) no monitor.

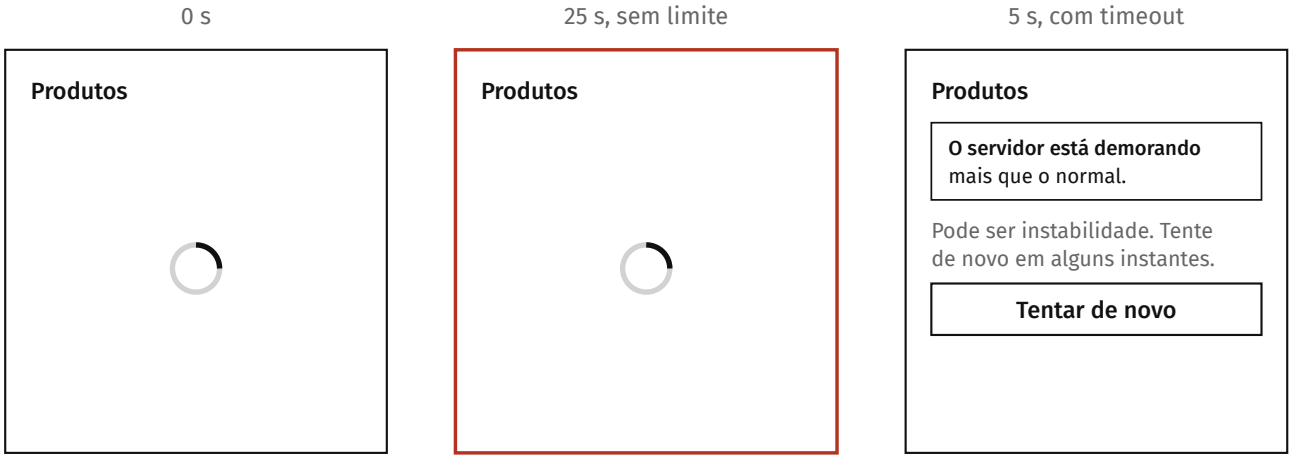


Figura 12.1. Sem limite, o spinner continua aos 25 s. Com timeout de 5 s, a tela desiste, explica e oferece outra tentativa.

12.2 Por que acontece

Servidores travam por motivos comuns: uma consulta lenta no banco, uma tabela bloqueada, um serviço externo que não responde. No celular, a conexão pode sumir sem ser fechada, e nenhum dos lados fica sabendo. Sem um limite no front, a tela espera o tempo que o pior desses casos durar, e passado o limite de dez segundos a pessoa já foi fazer outra coisa.

12.3 No front: desistir com explicação

`AbortSignal.timeout(5000)` cancela a requisição depois de cinco segundos, e o `fetch` falha com um erro de nome `TimeoutError`. Com isso, a tela consegue dizer a coisa certa: demorar demais é diferente de dar erro.

Frontend · Navegador

```
try {
  const resposta = await fetch('/api/produtos', { signal: AbortSignal.timeout(5000) })
  mostrarProdutos(await resposta.json())
} catch (erro) {
  if (erro.name === 'TimeoutError') {
    mostrarErro('O servidor está demorando mais que o normal.', tentarDeNovo)
  } else {
    mostrarErro('Não foi possível carregar os produtos.', tentarDeNovo)
  }
}
```

12.4 Na API: limites também

A API também não deve esperar para sempre. O Postgres aceita um `statement_timeout` que cancela consultas longas, e toda chamada a outro serviço deve ter seu próprio limite. Quando um deles estoura, a API responde rápido com `503` ou `504`, em vez de segurar a conexão até o front desistir. Um servidor que segura conexões travadas acaba sem conexões livres para quem chega depois.

12.5 Cuidados

Timeout curto demais derruba operações que são lentas de verdade, como um relatório pesado ou um upload. Essas precisam de progresso (tema 11, Upload com progresso e tempo real), não de um limite fixo. E o timeout cancela no front, mas não no servidor (o mesmo ponto do tema 8, Race conditions e AbortController): se a operação gravava algo, ela pode ter terminado depois que a tela desistiu.

O botão Tentar de novo resolve com um clique. O front também pode tentar sozinho, desde que saiba quando parar (tema 13, Retries e exponential backoff).

12.6 No seu projeto

Faça uma rota da sua API demorar, com um `sleep` numa rota de teste, e abra a tela que a usa. Depois de quanto tempo a tela desiste? Ela desiste?

Tema 13 · Frontend + API

Retries e exponential backoff

Retry é tentar de novo, automaticamente, quando uma requisição falha por um problema passageiro. Exponential backoff é esperar cada vez mais entre as tentativas, 1 s, depois 2 s, depois 4 s, para não piorar um servidor que já está com dificuldade.

13.1 O problema

Com a taxa de erro em 50%, a versão sem retry mostra erro em metade das vezes, mesmo que um segundo depois a API respondesse normalmente. A versão que tenta de novo sem parar faz o oposto: falhou, pergunta de novo na hora, para sempre. Com a API fora do ar, cada pessoa com a página aberta vira uma metralhadora de requisições, e a API, que talvez se recuperasse, não volta mais.

Na lojinha, o tema tem três versões: Sem retry, Retry sem parar e Exponential backoff. Use a taxa de erro em 50% ou 100%.

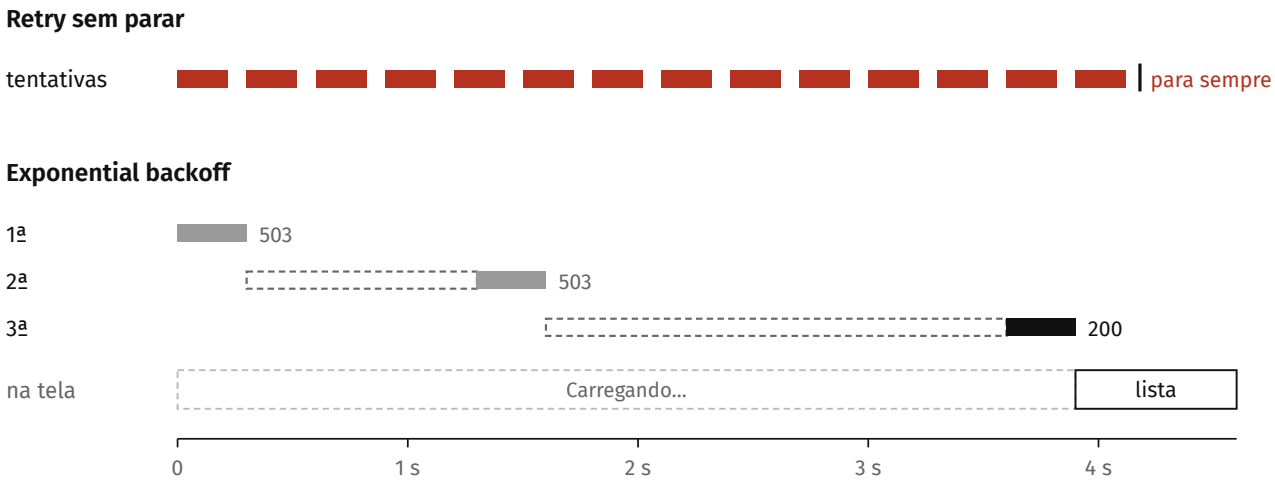


Figura 13.1. Em cima, tentativas sem pausa. Embaixo, três tentativas com esperas de 1 s e 2 s; a terceira dá certo.

13.2 Por que acontece

Muitas falhas são passageiras: um servidor reiniciando, um pico de acesso que faz a API responder `503`, a rede do celular piscando. Desistir na primeira transforma um soluço em erro. Tentar de novo na hora bate no servidor exatamente no pior momento, e quando milhares de clientes fazem isso juntos, a carga se multiplica.

13.3 No front: poucas tentativas, esperando o dobro

A cada falha passageira, a espera dobra; depois da última, a tela desiste e mostra o erro. Um pouco de aleatoriedade na espera, o *jitter*, evita que todos os clientes voltem no mesmo instante.

Frontend · Navegador

```
const ESPERAS = [1000, 2000, 4000]

async function buscarComRetry(url) {
  for (let tentativa = 0; ; tentativa++) {
    const resposta = await fetch(url).catch(() => null)        // null: falha de rede
    if (resposta?.ok) return resposta.json()

    const passageira = !resposta || resposta.status >= 500
    const espera = ESPERAS[tentativa]
    if (!passageira || espera === undefined) throw new Error('Não foi possível carregar')

    await new Promise((r) => setTimeout(r, espera + Math.random() * 300))
  }
}
```

13.4 Na API: dizer o que é passageiro

O front só sabe se vale tentar de novo pelo que a API responde. Erro da requisição é 4xx e não adianta repetir; falha passageira é 5xx, e uma API sobrecarregada pode responder `503` com o header `Retry-After`, dizendo quantos segundos esperar. Um `500` genérico para tudo, como no tema 2, Erros úteis, faz o front repetir requisições que nunca vão dar certo.

13.5 Cuidados

Retry só é seguro em operação que pode ser repetida. Buscar produtos pode repetir à vontade. Um `POST /pedidos` que falhou por timeout talvez tenha gravado o pedido antes da resposta se perder, e repeti-lo pode comprar duas vezes (tema 14, Clique duplo e idempotência).

13.6 No seu projeto

No DevTools, bloqueie uma URL da API (Network, botão direito, Block request URL) e desbloqueie alguns segundos depois. O front tentou de novo? Quantas vezes, e com que intervalo?

Tema 14 · Frontend + API

Clique duplo e idempotência

Uma operação é *idempotente* quando repeti-la dá o mesmo resultado que fazê-la uma vez. Comprar não é: dois pedidos iguais viram duas compras. O front diminui a chance de o pedido sair duas vezes; a API garante que, se sair, conta uma vez só.

14.1 O problema

Na lojinha, a pessoa clica em Comprar no Fone de ouvido Bluetooth. O pagamento leva um segundo, e nesse segundo a tela não muda. Ela clica de novo, e de novo. A tabela de pedidos mostra três pedidos, e o cartão foi cobrado três vezes em R$ 189,90.

Na lojinha, o tema tem três versões: Sem proteção, Botão travado e Chave de idempotência, com a tabela de pedidos embaixo do botão.

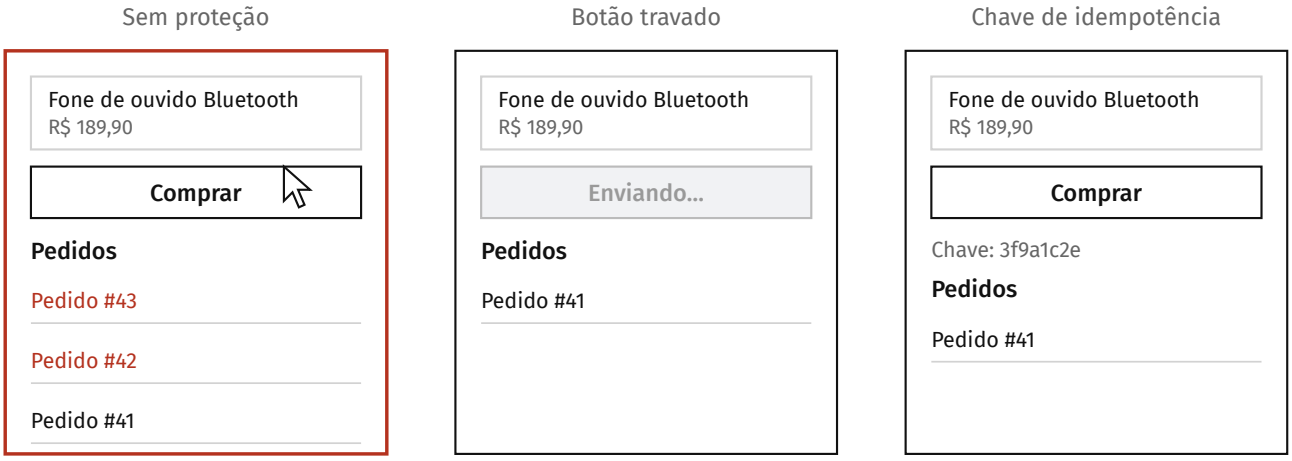


Figura 14.1. Três cliques em Comprar. Sem proteção, três pedidos; com o botão travado ou com a chave, um só.

14.2 Por que acontece

Espera sem sinal gera clique repetido, mas nem toda repetição vem de um clique. O retry automático (tema 13, Retries e exponential backoff) reenvia, a rede pode reenviar e a pessoa pode abrir duas abas. A API recebe duas requisições iguais e, sem nenhuma informação a mais, não tem como saber que é o mesmo pedido.

14.3 No front: travar o botão e mandar uma chave

Durante o envio, o botão mostra “Enviando...” e ignora novos cliques. Junto com o pedido vai uma chave de idempotência, gerada uma vez por intenção de compra: todas as tentativas desta compra mandam a mesma chave, e a próxima compra ganha uma chave nova.

Frontend · Navegador

```
let enviando = false
let chave = crypto.randomUUID()                    // uma chave por intenção de compra

async function comprar() {
  if (enviando) return                             // segundo clique: ignora
  enviando = true
  mostrarBotao('Enviando…', { travado: true })
  try {
    await fetch('/api/pedidos', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json', 'Idempotency-Key': chave },
      body: JSON.stringify({ produtoId: 1 }),
    })
    chave = crypto.randomUUID()                    // a próxima compra é outra intenção
  } finally {
    enviando = false
    mostrarBotao('Comprar')
  }
}
```

14.4 Na API: a mesma chave, o mesmo pedido

A coluna da chave é única no banco. Se a chave é nova, o pedido é criado; se já foi usada, nada é inserido e a API devolve o pedido que já existe, com a mesma resposta.

API · Servidor

```
const [novo] = await sql(
  `INSERT INTO pedidos (produto_id, chave_de_idempotencia) VALUES ($1, $2)
   ON CONFLICT (chave_de_idempotencia) DO NOTHING
   RETURNING id`,
  [produtoId, chave],
)
if (novo) return res.status(201).json(novo)

const [existente] = await sql(
  'SELECT id FROM pedidos WHERE chave_de_idempotencia = $1', [chave])
res.json(existente)                  // repetição: mesma resposta, nenhum pedido novo
```

14.5 Cuidados

O botão travado não basta sozinho: ele não protege contra retries, duas abas ou alguém chamando a API direto. Quem garante é a API. Existe uma proposta de padrão na IETF para o header `Idempotency-Key`, e serviços de pagamento como o Stripe já o usam assim.

Clique repetido de propósito, mil vezes por segundo, já não é distração: é abuso, e a API precisa de um limite (tema 15, Rate limiting).

14.6 No seu projeto

Com a rede em Slow 4G, clique três vezes rápido no botão que cria algo no seu sistema, como um pedido, um cadastro ou um comentário. Quantos registros apareceram no banco?

Tema 15 · API

Rate limiting

Rate limiting é limitar quantas requisições um cliente pode fazer num intervalo de tempo. Protege a API de abuso e de excesso, e quem passa do limite recebe um status próprio, o `429 Too Many Requests`.

15.1 O problema

A lojinha tem um campo de cupom. Sem limite, um script testa mil códigos por segundo até achar um que funcione. Com limite, mas sem explicação, a API bloqueia e o front mostra “Algo deu errado”; a pessoa de verdade, que só errou a digitação algumas vezes, não sabe se o cupom é inválido, se a loja caiu ou quando pode tentar de novo.

Na lojinha, o tema tem três versões: Sem limite, 429 sem explicação e Com Retry-After. O limite é de 5 tentativas a cada 10 s.

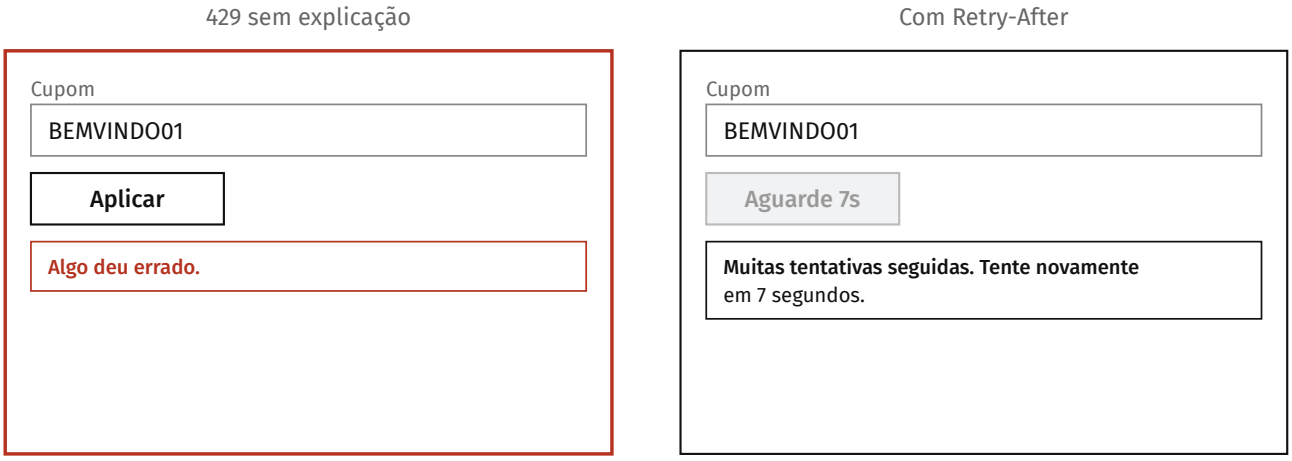


Figura 15.1. A mesma sexta tentativa em dez segundos. Com o Retry-After, a tela sabe quanto falta.

15.2 Por que acontece

Sem limite, qualquer rota que faz algo caro ou sensível, como login, cupom ou envio de SMS, vira alvo. Com o limite mal comunicado, quem paga é o usuário legítimo: a API sabe exatamente quando ele poderá tentar de novo, mas não conta.

15.3 Na API: contar e responder 429 com Retry-After

O rate limiter conta as requisições de cada cliente numa janela de tempo. Passou do máximo, responde `429` e diz no header `Retry-After` quantos segundos faltam para a janela zerar.

API · Servidor

```
const MAXIMO = 5, JANELA = 10_000
const contagens = new Map()                       // ip -> { quantidade, zeraEm }

function limitar(req, res, next) {
  const agora = Date.now()
  let c = contagens.get(req.ip)
  if (!c || agora >= c.zeraEm) {
    c = { quantidade: 0, zeraEm: agora + JANELA }
    contagens.set(req.ip, c)
  }

  if (++c.quantidade > MAXIMO) {
    const segundos = Math.ceil((c.zeraEm - agora) / 1000)
    res.set('Retry-After', String(segundos))
    const erro = `Muitas tentativas. Tente novamente em ${segundos} segundos.`
    return res.status(429).json({ erro })
  }
  next()
}

app.post('/api/cupons', limitar, validarCupom)
```

No front, o `429` vira uma espera visível: o botão fica travado com "Aguarde 7s", contando para baixo, até o tempo do `Retry-After` acabar.

15.4 Cuidados

Contar em memória só funciona com um servidor; com vários, a contagem precisa ficar num lugar comum, como o Redis. Limitar por IP também pega muita gente junta: uma faculdade inteira pode sair para a internet pelo mesmo IP. Quando houver login, limite por usuário. Os headers `RateLimit-Limit` e `RateLimit-Remaining`, em padronização na IETF, permitem ao front mostrar quantas tentativas restam antes do bloqueio.

Outro bloqueio que a API devolve no meio do uso é o da sessão que expirou, e ele também não pode custar o trabalho da pessoa (tema 16, Sessão expirada).

15.5 No seu projeto

Mande 50 requisições seguidas para o login da sua API, com um loop no terminal. Em que momento ela para de aceitar? A resposta diz quando tentar de novo?

Tema 16 · Frontend + API

Sessão expirada

Sessões expiram por segurança. O problema é quando isso acontece no meio de um formulário longo: a API recusa o envio com `401`, e se o front reage mandando para a tela de login, tudo o que foi digitado se perde.

16.1 O problema

Na lojinha, um cadastro longo pede nome, e-mail, telefone, endereço e observações. A pessoa preenche tudo, sai para pegar um café e volta. A sessão expirou nesse meio-tempo. Ela clica em Salvar, é levada para o login, entra e volta para um formulário vazio.

Na lojinha, o tema tem duas versões: Perde o formulário e Modal de login. O botão "Expirar sessão agora" fica no monitor.

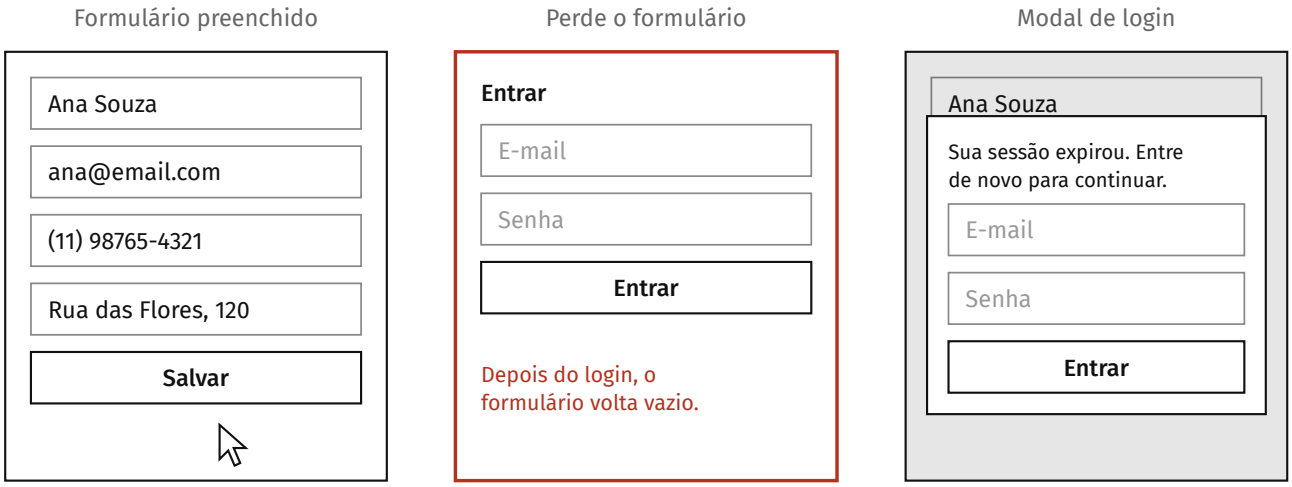


Figura 16.1. O mesmo Salvar com a sessão expirada. O modal pede o login por cima, e o formulário continua embaixo, preenchido.

16.2 Por que acontece

A API fez a parte dela: respondeu `401`, que quer dizer "não sei quem você é". Quem perdeu os dados foi o front, que tratou o `401` como fim de linha e trocou de página. Redirecionar desmonta a tela, e o estado em memória vai junto.

16.3 Na API: 401 claro

Sessão ausente ou expirada é `401`, com uma mensagem que diga isso; `403` é para quem está logado mas não tem permissão. A diferença importa porque o front reage de jeitos diferentes: `401` pede um novo login, `403` não. O token da sessão vai num cookie `httpOnly`, que o JavaScript da página não consegue ler.

16.4 No front: pedir login por cima e reenviar

Ao receber `401`, o front guarda o que ia ser enviado e abre o login num modal, por cima do formulário. Depois do login, refaz o envio sozinho, sem a pessoa clicar em Salvar de novo.

Frontend · Navegador

```
let pendente

async function salvar(cadastro) {
  const resposta = await fetch('/api/cadastros', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(cadastro),
  })

  if (resposta.status === 401) {
    pendente = cadastro                          // guarda o que ia ser enviado
    abrirModalDeLogin()                          // o formulário continua na tela
    return
  }
  mostrarSucesso(await resposta.json())
}

async function aoEntrar() {
  fecharModalDeLogin()
  if (pendente) await salvar(pendente)           // refaz o envio que tinha falhado
}
```

16.5 Cuidados

O modal salva o caso em que a página continua aberta. Para sobreviver a um F5 ou a uma aba fechada, o rascunho precisa ir para o `localStorage` enquanto a pessoa digita, com cuidado para não guardar dado sensível. Muitos sistemas também renovam a sessão sozinhos, com um *refresh token*, e só pedem login quando ele também expirou.

Guardar o que a pessoa fez para enviar depois é o mesmo recurso que salva a ação quando a conexão cai (tema 17, Offline e conexão instável).

16.6 No seu projeto

Preencha um formulário, apague o cookie de sessão no DevTools (Application, Cookies) e envie. O que acontece com o que você digitou?

Tema 17 · Frontend + API

Offline e conexão instável

A conexão cai no metrô, no elevador, no Wi-Fi da faculdade. Uma tela preparada avisa que está sem conexão, guarda o que a pessoa fez e envia sozinha quando a rede volta, em vez de mostrar “Algo deu errado” e esquecer a ação.

17.1 O problema

No metrô, a pessoa abre a lojinha e favorita três livros para ver depois. Na versão ruim, cada clique mostra “Algo deu errado” e o coração não fica marcado. Ela tenta de novo, recebe o mesmo erro e desiste. Quando chega em casa, os favoritos não estão lá.

Na lojinha, o tema tem duas versões: Perde a ação e Fila offline. A queda é simulada com o modo Offline do DevTools, só na aba da lojinha; o monitor, em outra aba, continua mostrando tudo.

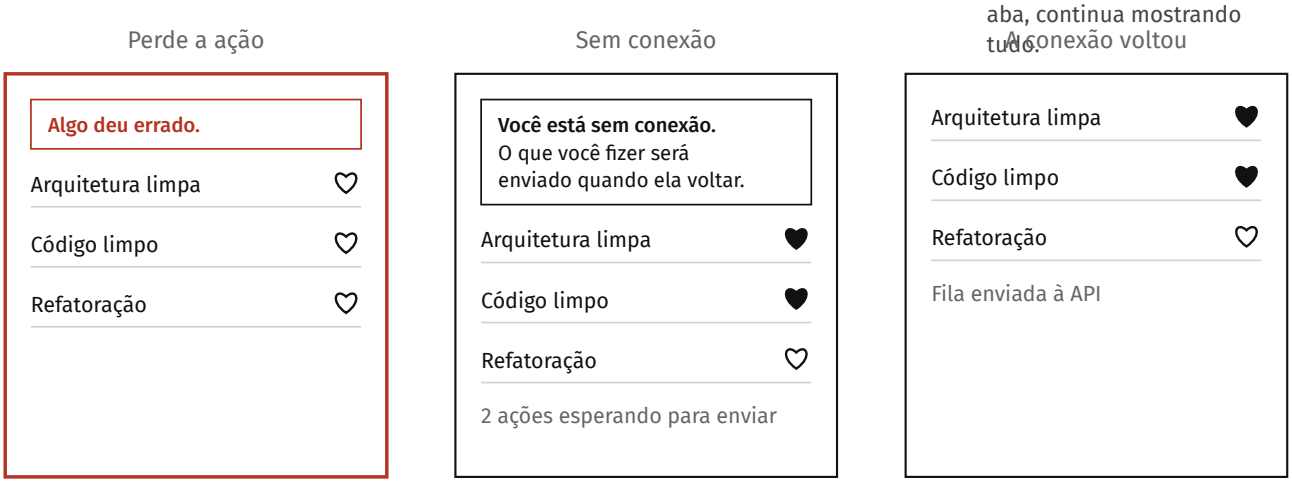


Figura 17.1. Sem conexão, a versão boa avisa, marca na hora e guarda as ações. Quando a rede volta, a fila é enviada.

17.2 Por que acontece

Sem rede, o `fetch` falha com um `TypeError`, e a ação existia só dentro da chamada que falhou. Não há aviso de que a conexão caiu, então a pessoa não sabe se o problema é dela ou da loja. O navegador até avisa: `navigator.onLine` diz o estado atual, e os eventos `offline` e `online` disparam quando ele muda.

17.3 No front: avisar, enfileirar e enviar quando voltar

Com ou sem conexão, o caminho é o mesmo: a tela muda na hora, como no Optimistic UI, e a ação entra numa fila guardada no `localStorage`, que sobrevive até a aba fechada. A fila é enviada em ordem, e cada ação

só sai dela depois que a API confirma. As funções `lerFila` e `salvarFila` só leem e gravam essa lista.

Frontend · Navegador

```
function alternarFavorito(produtoId, favoritar) {
  marcarNaTela(produtoId, favoritar)
  salvarFila([...lerFila(), { produtoId, favoritar }])
  enviarFila()
}

async function enviarFila() {
  for (let fila = lerFila(); fila.length > 0 && navigator.onLine; fila = lerFila()) {
    const [acao] = fila
    const metodo = acao.favoritar ? 'PUT' : 'DELETE'
    await fetch(`/api/favoritos/${acao.produtoId}`, { method: metodo })
    salvarFila(lerFila().slice(1))              // só sai da fila depois da resposta
  }
}

window.addEventListener('online', enviarFila)
window.addEventListener('offline', () => mostrarAviso('Você está sem conexão'))
```

17.4 Na API: aguentar a repetição

A conexão pode cair depois de a API gravar e antes de a resposta chegar. Nesse caso, a ação continua na fila e vai de novo. Isso só é seguro porque favoritar é `PUT` e desfavoritar é `DELETE`, que são idempotentes: chegar duas vezes dá no mesmo. Para uma ação que cria algo, como um `POST`, a fila precisaria mandar uma chave de idempotência (tema 14, Clique duplo e idempotência).

17.5 Cuidados

`navigator.onLine` verdadeiro só garante que existe uma rede, não que a API responde; por isso a ação vai para a fila mesmo quando parece haver conexão. Uma ação enfileirada também pode ficar velha, como favoritar um produto que saiu da loja: a API deve responder algo que o front saiba tratar, e o front tira a ação da fila e avisa. E nem tudo deve esperar a rede voltar; um pagamento não.

Do lado da leitura, mostrar o que já se tem enquanto a rede não responde é o assunto do próximo bloco (tema 18, Caching).

17.6 No seu projeto

Coloque o DevTools em Offline, faça uma ação no seu sistema e volte para Online. A ação chegou à API? E se você fechar a aba enquanto estiver offline?

Bloco 4

Rapidez de verdade

Ficar rápido quase nunca é trocar de servidor. É não buscar de novo o que já se tem, não baixar o que ninguém vai ver e fazer a API entregar exatamente o que a tela precisa.

18	Caching	Frontend + API
19	Imagens com signed URL e cache	Frontend + API
20	Lazy loading	Frontend
21	N+1 requests	API
22	Paginação e infinite scroll	Frontend + API

Tema 18 · Frontend + API

Caching

Cache é guardar uma resposta para não buscar de novo o que acabou de chegar. Pode morar no front, em memória ou numa biblioteca como o TanStack Query, no navegador, pelo cache HTTP que a API controla com headers, ou nos dois. O cuidado é não mostrar um dado velho como se fosse atual.

18.1 O problema

Na lojinha, a pessoa olha Eletrônicos, passa para Livros e volta para Eletrônicos. Sem cache, a lista some e carrega tudo de novo, embora ela tenha visto a mesma lista três segundos antes. Em rede lenta, cada troca de categoria é uma espera.

Na lojinha, o tema tem quatro versões: Sem cache, Cache na mão, TanStack Query e Cache HTTP.

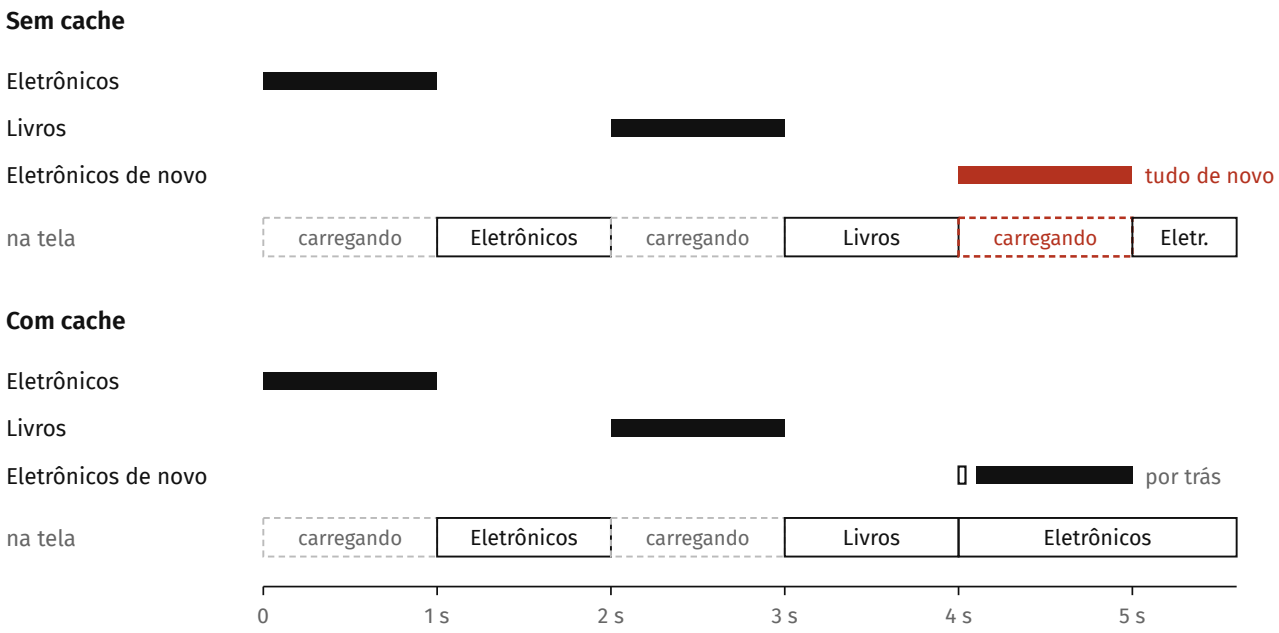


Figura 18.1. Ida e volta entre duas categorias. Com cache, a volta a Eletrônicos aparece na hora, e a versão nova chega por trás.

18.2 Por que acontece

Quando cada tela busca seus dados ao aparecer e descarta tudo ao sair, a resposta anterior vai embora junto. A volta é tratada como a primeira visita.

18.3 No front: mostrar o que tem e atualizar por trás

A ideia tem nome, *stale-while-revalidate*: mostrar na hora o que está guardado, mesmo que um pouco velho, e buscar a versão nova em background. No TanStack Query, isso vem pronto com a `queryKey`, e o `staleTime` diz por quanto tempo o dado nem precisa ser buscado de novo.

Frontend · Navegador

```
const cache = new Map()            // fora da tela: sobrevive à troca de categoria

async function mostrarCategoria(categoria) {
  const guardados = cache.get(categoria)
  if (guardados) mostrarProdutos(guardados)         // 1. na hora, o que já tem

  const novos = await buscarProdutos(categoria)     // 2. a versão nova, por trás
  cache.set(categoria, novos)
  if (categoria === categoriaAtual) mostrarProdutos(novos)
}
```

18.4 Na API: cache HTTP

A API pode dizer ao navegador como guardar a resposta. `Cache-Control: max-age=10` faz o navegador usar a cópia dele por dez segundos sem nem perguntar. Depois disso, ele pergunta se mudou, mandando a `ETag` que guardou; se não mudou, a resposta é `304`, sem corpo.

API · Servidor

```
app.get('/api/produtos-com-cache', async (req, res) => {
  const corpo = JSON.stringify(await listarProdutos(req.query.categoria))

  res.set('Cache-Control', 'max-age=10')              // 10 s sem nem perguntar
  res.set('ETag', `"${hash(corpo)}"`)                 // depois, pergunta: mudou?
  if (req.fresh) return res.status(304).end()         // não mudou: 304, sem corpo

  res.type('json').send(corpo)
})
```

18.5 Cuidados

Dado que muda a todo instante ou que é pessoal, como saldo e carrinho, pede `max-age` curto, `private` ou `no-store`. Depois de uma escrita, o cache do que mudou precisa ser descartado: cadastrou um produto, a lista guardada ficou velha. E o cache do navegador não pode ser apagado pelo servidor; um `max-age` longo num dado que muda é um erro difícil de desfazer.

Imagens são o caso em que o cache mais pesa, e um detalhe das signed URLs costuma anulá-lo (tema 19, Imagens com signed URL e cache).

18.6 No seu projeto

Com a aba Network aberta, vá de uma tela para outra e volte. A lista foi baixada de novo? Veja também se as respostas da sua API têm `Cache-Control` e `ETag`.

Tema 19 · Frontend + API

Imagens com signed URL e cache

Signed URL é um link temporário para um arquivo privado guardado num storage como o S3 ou o MinIO: a assinatura e a validade vão na própria URL. Bem usada, protege as imagens sem impedir o cache; mal usada, faz o navegador baixar todas as fotos de novo a cada visita.

19.1 O problema

A vitrine da lojinha mostra doze produtos com foto. Na versão ruim, a cada F5 as doze fotos são baixadas de novo, embora o arquivo seja o mesmo. A vitrine pisca e demora toda vez, e no celular o plano de dados paga por imagens que o navegador já tinha.

Na lojinha, o tema tem duas versões: URL nova a cada vez e URL reaproveitada. Dê F5 com a aba Network aberta.

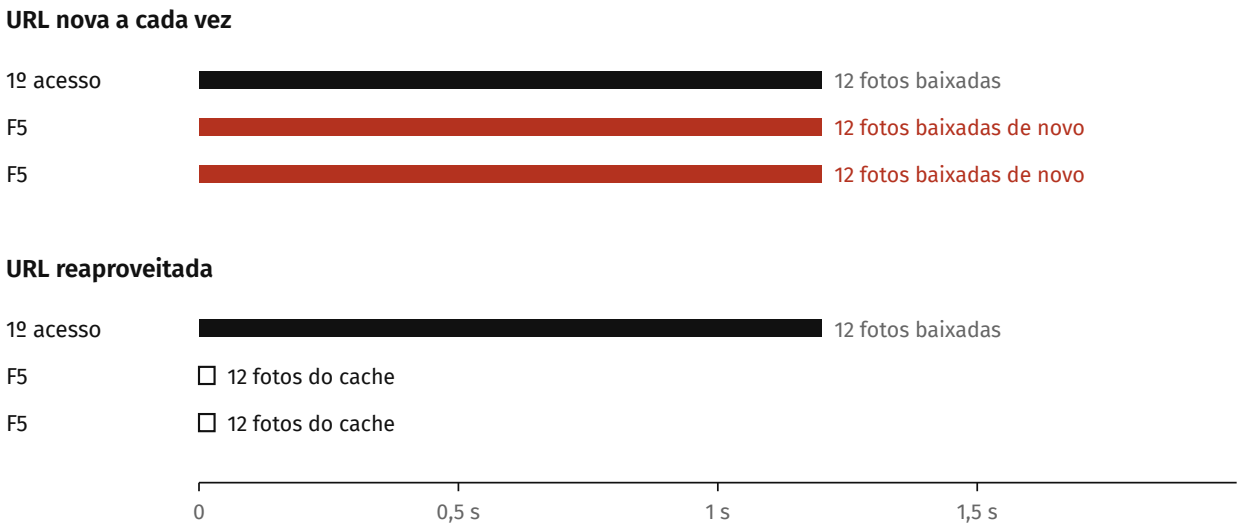


Figura 19.1. Três acessos à vitrine. Com a URL reaproveitada, o F5 usa as fotos que o navegador já tem.

19.2 Por que acontece

Para o navegador, URL diferente é arquivo diferente. A signed URL leva a data em que foi assinada e a assinatura na query string, e se a API assina na hora a cada requisição, cada F5 traz URLs novas para as mesmas fotos:

Na rede

```
/fotos/5.jpg?X-Amz-Date=20261018T100215Z&X-Amz-Expires=3600&X-Amz-Signature=9f2c…
/fotos/5.jpg?X-Amz-Date=20261018T100348Z&X-Amz-Expires=3600&X-Amz-Signature=41ab…
```

19.3 Na API: a mesma URL durante um período

Em vez de assinar com a hora exata, a API assina como se fosse o começo da hora. Durante a hora inteira, a URL de cada foto é idêntica, e o navegador usa a que já tem. A validade cobre o começo da hora mais uma hora de folga, para nunca expirar enquanto a foto estiver no cache.

API · Servidor

```
const UMA_HORA = 60 * 60 * 1000
const comecoDaHora = new Date(Math.floor(Date.now() / UMA_HORA) * UMA_HORA)

const comando = new GetObjectCommand({ Bucket: 'fotos', Key: '5.jpg' })
const foto = await getSignedUrl(s3, comando, {
  signingDate: comecoDaHora,          // a mesma durante a hora: a URL não muda
  expiresIn: 2 * 60 * 60,             // vale 2 h a partir do começo da hora
})
```

Para o navegador guardar a imagem, o arquivo também precisa de `Cache-Control`, como `max-age=3600`, definido no objeto do bucket. No front, nada muda: a tag `<img>` recebe a URL e o navegador faz o resto. Só não acrescente parâmetros como `?t=` com a hora atual, que anulam o cache do mesmo jeito.

19.4 Cuidados

O cache nunca pode durar mais que a URL: se a assinatura expira antes, a próxima requisição da imagem falha. Outra saída é guardar a signed URL no servidor, em memória ou no Redis, e devolver a mesma até perto de expirar.

Com o cache resolvido para quem volta, falta não baixar à toa, na primeira visita, o que a pessoa nem vai ver (tema 20, Lazy loading).

19.5 No seu projeto

Com “Disable cache” desmarcado na aba Network, abra uma página com imagens privadas e dê F5. As imagens aparecem como vindas do cache ou baixam de novo? Compare as URLs entre dois F5.

Tema 20 · Frontend

Lazy loading

Lazy loading é carregar uma coisa só quando ela for necessária: imagens quando estão chegando perto da tela, o código de uma tela quando alguém a abre. O primeiro carregamento fica menor e chega antes.

20.1 O problema

A página do tema na lojinha tem 200 imagens. Na versão ruim, ao abrir a página o navegador baixa as 200, embora só umas seis caibam na tela. Quem olha as primeiras e sai pagou por todas, e as imagens do topo disputam a conexão com as 194 que ninguém viu.

Na lojinha, o tema tem duas versões: Tudo de uma vez e loading="lazy", com um contador de imagens e megabytes baixados no topo.

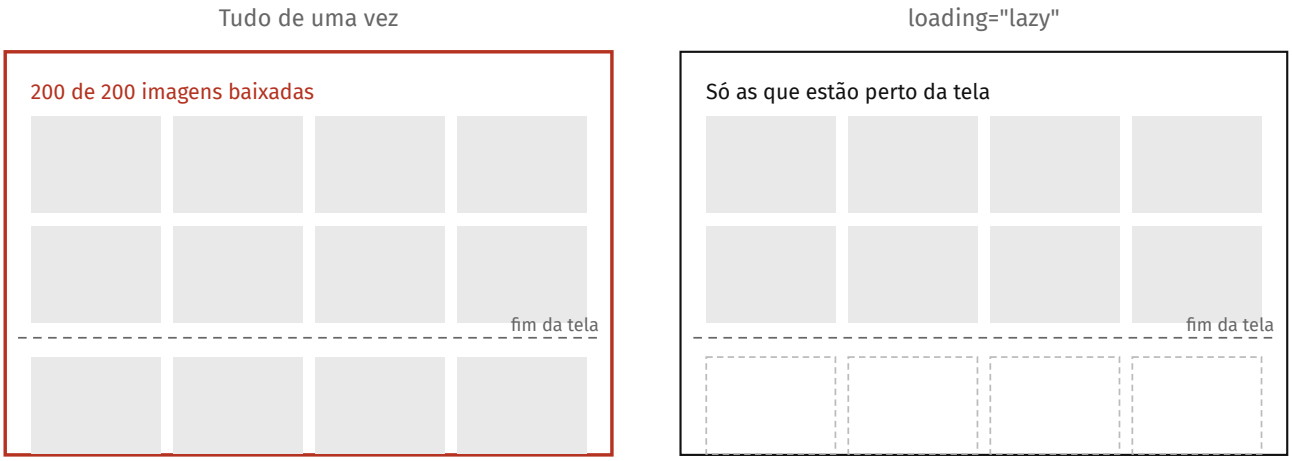


Figura 20.1. A mesma página logo depois de abrir, sem rolar. O contador mostra quantas imagens já foram baixadas.

20.2 Por que acontece

Por padrão, uma `<img>` é baixada assim que o navegador lê o HTML, esteja ela no topo ou no fim da página. O mesmo vale para o JavaScript: se o código de todas as telas está num arquivo só, quem abre a página inicial baixa também o código da tela de relatórios.

20.3 No front: só quando aparecer

Para imagens, o atributo `loading="lazy"` resolve sem nenhum JavaScript: o navegador só baixa quando a imagem está chegando perto da área visível. Para código, o `import()` dinâmico separa uma tela num arquivo à parte, baixado quando ela abre.

Frontend · HTML

```html
<img src="/api/imagens/37" alt="Garrafa térmica" loading="lazy" width="640" height="480">
```

Frontend · Navegador

```
// O código da tela de relatórios só é baixado quando alguém abre a tela
const Relatorios = lazy(() => import('./Relatorios'))
```

20.4 Cuidados

`width` e `height` são obrigatórios: sem eles, todas as imagens começam com altura zero, cabem juntas na tela e o navegador baixa todas (tema 6, Layout shift). Não use lazy na imagem principal do topo, que é a primeira coisa que a pessoa vê; atrasá-la piora justamente o que mais importa. E "perto da tela" é generoso: o Chrome começa a baixar centenas de pixels antes, para a imagem já estar lá quando a rolagem chegar.

Baixar menos também vale para dados. Às vezes o problema não é o tamanho, mas quantas requisições a tela faz para se montar (tema 21, N+1 requests).

20.5 No seu projeto

Na aba Network, filtre por Img e abra a página sem rolar. Quantas imagens foram baixadas? Quantas estão visíveis?

Tema 21 · API

N+1 requests

N+1 é quando montar uma tela exige uma requisição para o item principal e mais uma para cada item relacionado: uma para o produto, N para os comentários. O nome vem do banco de dados, onde o mesmo padrão aparece nas consultas.

21.1 O problema

A página do Monitor 24 polegadas mostra o produto, o vendedor e doze comentários. Na versão ruim, a API tem um endpoint para cada coisa e a tela junta as peças: uma requisição para o produto, uma para o vendedor, uma para a lista de comentários e uma para cada comentário. São 15 requisições, em três ondas, porque o vendedor só pode ser pedido depois de se saber quem ele é.

Na lojinha, o tema tem duas versões: Um endpoint por recurso e Um endpoint por tela. No monitor, 15 requisições contra 1.

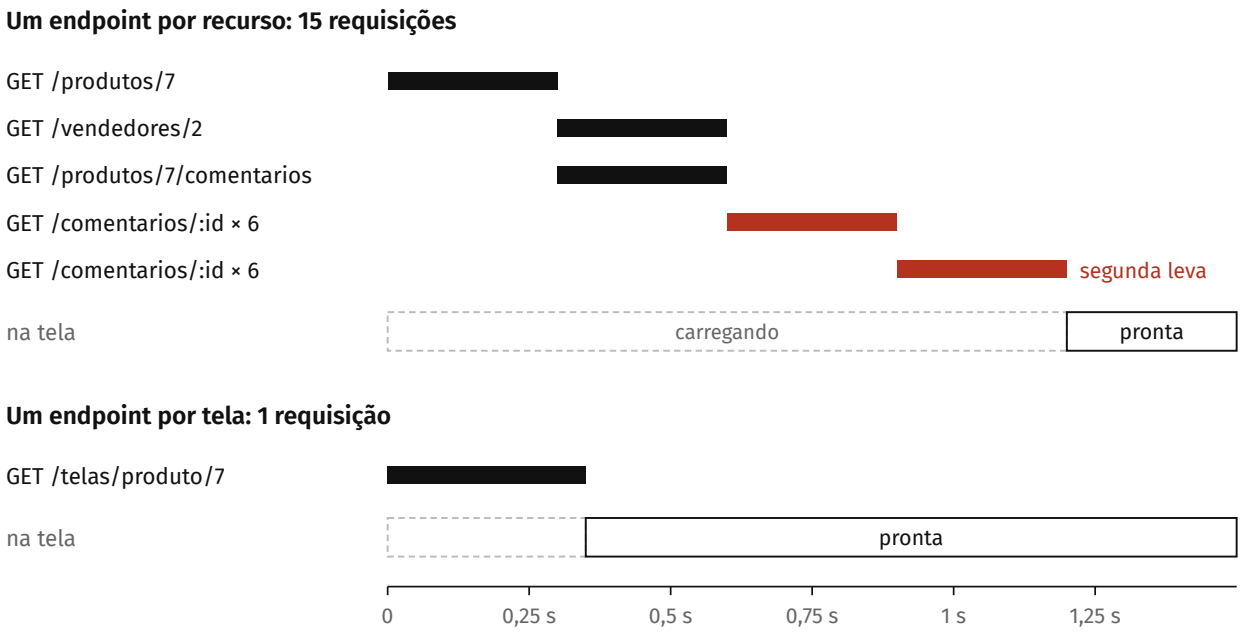


Figura 21.1. A mesma página do produto. Em cima, três ondas de requisições; embaixo, uma resposta com tudo.

21.2 Por que acontece

A API foi desenhada pensando nas tabelas, um recurso por endpoint, e não nas telas que vão usá-la. Cada requisição paga a latência inteira da rede, e as que dependem de outra esperam em fila. No HTTP/1.1, o navegador ainda abre no máximo seis conexões por servidor, e os doze comentários saem em duas levas.

21.3 Na API: um endpoint que entrega o que a tela precisa

Um endpoint por tela devolve tudo numa resposta. Dentro dele, o cuidado é o mesmo no banco: um `JOIN` traz produto e vendedor juntos, e uma consulta só traz todos os comentários. Fazer um `SELECT` por comentário dentro da API só mudaria o N+1 de lugar.

API · Servidor

```
// GET /api/telas/produto/7 - a página inteira numa resposta
app.get('/api/telas/produto/:id', async (req, res) => {
  const [produto] = await sql(
    `SELECT p.id, p.nome, p.preco_centavos,
            json_build_object('nome', v.nome, 'cidade', v.cidade) AS vendedor
     FROM produtos p JOIN vendedores v ON v.id = p.vendedor_id
     WHERE p.id = $1`,
    [req.params.id],
  )
  const comentarios = await sql(
    'SELECT autor, texto, nota FROM comentarios WHERE produto_id = $1',
    [req.params.id],
  )
  res.json({ ...produto, comentarios })
})
```

21.4 Cuidados

Endpoint por tela acopla a API à tela: mudou a tela, muda a API. Padrões como o BFF (Backend for Frontend) e ferramentas como o GraphQL são formas organizadas de lidar com isso. E entregar o que a tela precisa também quer dizer não entregar demais: se o produto tivesse dez mil comentários, eles viriam aos pedaços (tema 22, Paginação e infinite scroll).

21.5 No seu projeto

Abra a tela mais importante do seu sistema com a aba Network aberta. Quantas requisições ela faz até ficar pronta? Quantas esperam outra terminar?

Tema 22 · Frontend + API

Paginação e infinite scroll

Paginar é entregar uma lista em pedaços. Páginas numeradas e infinite scroll são dois jeitos de mostrar esses pedaços na tela; offset e cursor são dois jeitos de a API saber qual pedaço mandar.

22.1 O problema

O feed da lojinha tem 5.000 publicações. Sem paginação, a API manda todas de uma vez: a resposta é grande, o banco lê tudo, a tela demora para desenhar, e a pessoa olha as dez primeiras.

Paginar resolve isso, mas cria outro problema quando a lista muda. Com infinite scroll por offset, a pessoa vê as dez primeiras publicações; enquanto ela lê, alguém publica uma nova no topo. Quando a rolagem pede a próxima página, a publicação 4991, que era a décima, virou a décima primeira, e aparece de novo.

Na lojinha, o tema tem quatro versões: Sem paginação, Páginas numeradas, Infinito com offset e Infinito com cursor. O botão “Simular: alguém publicou agora” insere uma publicação no topo.

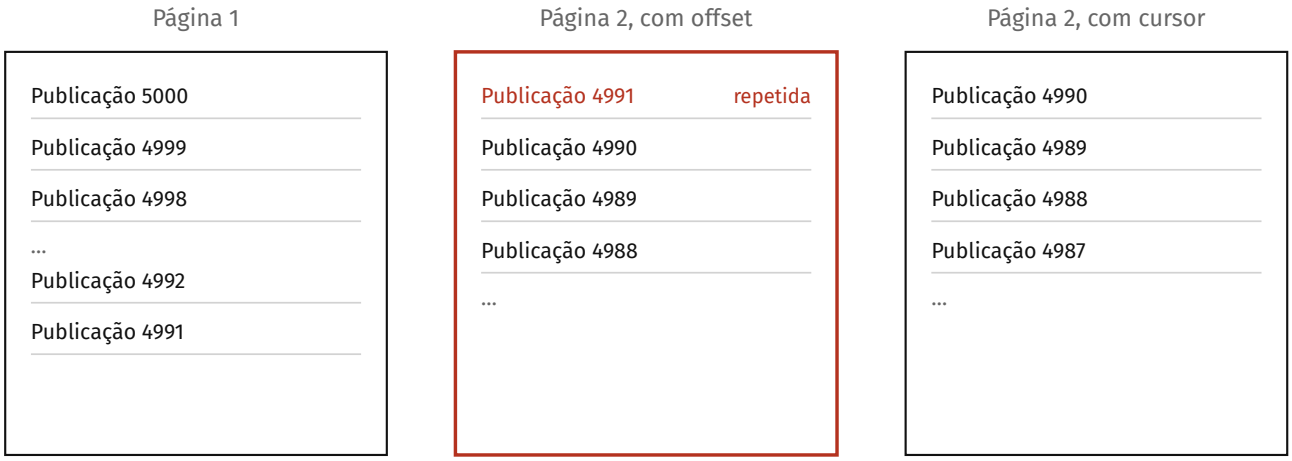


Figura 22.1. Entre a primeira e a segunda página, alguém publicou a 5001. Com offset, a 4991 aparece duas vezes; com cursor, a lista segue certa.

22.2 Por que acontece

O offset conta posições: “pule as dez primeiras”. Quando um item entra ou sai, as posições mudam e a conta erra, repetindo ou pulando itens. O cursor aponta para um item: “os dez que vêm depois da 4991”. O item não muda de lugar quando outro entra. Offset grande ainda é lento, porque o banco lê e descarta todas as linhas que pulou.

22.3 Na API: paginar por cursor

API · Servidor

```
// GET /api/feed/cursor?depoisDe=4991 - os 10 que vêm depois da publicação 4991
app.get('/api/feed/cursor', async (req, res) => {
  const depoisDe = req.query.depoisDe ? Number(req.query.depoisDe) : null

  const itens = await sql(
    `SELECT id, texto FROM publicacoes
     WHERE ($1::int IS NULL OR id < $1)
     ORDER BY id DESC LIMIT 10`,
    [depoisDe],
  )
  const proximoCursor = itens.length === 10 ? itens.at(-1).id : null   // null: acabou
  res.json({ itens, proximoCursor })
})
```

22.4 No front: páginas ou infinite scroll

Páginas numeradas servem para procurar e comparar: resultados de busca, tabelas, listas em que a pessoa quer voltar a um item ou saber quantos são. Combinam com a página na URL (tema 3, URL como estado). O infinite scroll serve para consumir sem um objetivo definido, como um feed. Ele dificulta chegar ao rodapé e voltar ao ponto onde se estava. No front, um `IntersectionObserver` avisa quando o fim da lista aparece na tela, e só então a próxima página é pedida.

Frontend · Navegador

```
const observador = new IntersectionObserver(async ([fim]) => {
  if (!fim.isIntersecting || acabou) return
  const resposta = await fetch(`/api/feed/cursor?depoisDe=${cursor ?? ''}`)
  const { itens, proximoCursor } = await resposta.json()
  adicionarNaLista(itens)
  cursor = proximoCursor
  acabou = proximoCursor === null
})
observador.observe(document.querySelector('#fim-da-lista'))
```

22.5 Cuidados

Com cursor, não dá para pular direto para a página 37; para isso, páginas numeradas com offset continuam sendo o caminho, aceitando o risco de repetição. A ordenação do cursor precisa ser única e estável, como o id, ou a data mais o id. E paginar bem fecha o que o tema 21, N+1 requests, começou: a API entrega pouco e certo, e a tela mostra rápido.

22.6 No seu projeto

Procure no seu código uma consulta sem `LIMIT` que alimenta uma lista na tela. Quantas linhas ela devolve hoje? E daqui a um ano?

Para levar

Checklist para o seu projeto

Os 22 temas em forma de pergunta. Marque o que o seu projeto já faz; o que ficar sem marca tem um tema inteiro explicando o caminho.

1. Antes do clique

- ☐ 1 O formulário mostra o que falta antes de ir para o servidor, e nada do que foi digitado se perde num erro?
- ☐ 2 Quando a API recusa um dado, ela responde com o status certo e diz qual campo está errado?
- ☐ 3 Filtros, busca e página atual sobrevivem a um F5 e vão junto quando o link é copiado?

2. Durante a espera

- ☐ 4 Cada tela que busca dados tem desenho próprio para carregando, erro (com “Tentar de novo”) e vazio (com orientação)?
- ☐ 5 Enquanto carrega, a tela mostra o formato do que vai aparecer?
- ☐ 6 Imagens, banners e avisos têm espaço reservado, para nada mudar de lugar quando chegam?
- ☐ 7 Buscas e eventos em rajada, como digitação, scroll e resize, passam por debounce ou throttle?
- ☐ 8 Quando a pessoa troca rápido de aba ou de filtro, a requisição anterior é cancelada ou descartada?
- ☐ 9 Ações pequenas e reversíveis mudam a tela na hora e voltam com aviso se a API recusar?
- ☐ 10 Toda ação tem uma confirmação visível que não trava a tela?
- ☐ 11 Uploads mostram o progresso do envio e do processamento?

3. Quando dá errado

- ☐ 12 Toda requisição tem um tempo máximo, e a tela explica e oferece tentar de novo quando ele estoura?
- ☐ 13 Falhas passageiras são repetidas poucas vezes, com espera crescente, e só em operações seguras de repetir?

- [ ] 14 Um pedido enviado duas vezes, por clique duplo, retry ou duas abas, vira um pedido só?
- [ ] 15 Rotas sensíveis têm limite de requisições, e o 429 diz quando tentar de novo?
- [ ] 16 Se a sessão expira no meio de um formulário, a pessoa entra de novo sem perder o que digitou?
- [ ] 17 Sem conexão, a tela avisa, guarda as ações e as envia quando a rede volta?

4. Rapidez de verdade

- [ ] 18 Voltar a uma tela já vista mostra os dados na hora e atualiza por trás?
- [ ] 19 Imagens privadas continuam vindo do cache depois de um F5?
- [ ] 20 Imagens fora da tela e o código de telas pouco usadas só são baixados quando necessário?
- [ ] 21 A tela principal se monta com poucas requisições, sem uma por item?
- [ ] 22 Toda lista tem limite na API, e a infinite scroll não repete nem pula itens?

Para levar

Como rodar a lojinha

A lojinha tem um front em React com Vite e Mantine, uma API em Node com Express e TypeScript, um banco Postgres e um MinIO, que faz o papel do S3 para as fotos. O banco e o MinIO rodam no Docker; o resto roda direto na sua máquina.

Requisitos

Node 22 ou mais novo, Git e o Docker Desktop aberto.

Passo a passo

Terminal

```
git clone https://github.com/adriangpo/lojinha-ux
cd lojinha-ux

npm run instalar     # só na primeira vez
npm run banco        # sobe o Postgres (porta 5433) e o MinIO (9000)
npm run dev          # sobe a API (3333) e o front (5173)
```

Com tudo no ar, a lojinha abre em `http://localhost:5173` e o monitor em `http://localhost:5173/monitor`. Abra os dois em abas separadas. Para zerar o banco, rode `docker compose down -v` e depois `npm run banco` de novo; as fotos voltam sozinhas quando a API sobe.

Usando a lojinha e o monitor

O menu da lojinha lista os 22 temas na ordem desta apostila. Em cada página, o seletor no canto superior direito troca entre as versões, da ruim para a boa, e o botão “Ver código” mostra o arquivo daquela versão.

O monitor mostra o que acontece entre a pessoa, a tela, o navegador, a API e o banco, em barras no tempo ou num diagrama de sequência. Clicando numa barra, aparecem o status, os headers que importam, o body da requisição e da resposta e o SQL executado. O painel do monitor também controla a lojinha:

Controle	Para quê
Latência: Nenhuma, 300 ms, 1 s, 3 s, Aleatória ou Travada	Deixar a API lenta, irregular ou parada (temas 5, 6, 9 e 12)
Taxa de erro: 0%, 25%, 50% ou 100%	Fazer a API responder 503 (temas 4, 9 e 13)
Expirar sessão agora	Derrubar a sessão no meio do formulário (tema 16, Sessão expirada)
Fixar para comparar	Guardar a linha do tempo de uma versão e rodar a outra embaixo

Para comparar duas versões, rode a ruim, clique em Fixar para comparar e rode a boa: as duas linhas do tempo ficam uma embaixo da outra, cada uma com o seu placar de requisições.

Para levar

Referências

Jakob Nielsen. Response Times: The 3 Important Limits. Nielsen Norman Group, 1993.
nngroup.com/articles/response-times-3-important-limits

GOV.UK Design System. Button, seção sobre botões desabilitados.
design-system.service.gov.uk/components/button

RFC 9110. HTTP Semantics: métodos, idempotência, status codes e Retry-After.
rfc-editor.org/rfc/rfc9110

RFC 9457. Problem Details for HTTP APIs.
rfc-editor.org/rfc/rfc9457

RFC 6585. Additional HTTP Status Codes, onde está o 429.
rfc-editor.org/rfc/rfc6585

RFC 9111. HTTP Caching.
rfc-editor.org/rfc/rfc9111

RFC 5861. HTTP Cache-Control Extensions for Stale Content (stale-while-revalidate).
rfc-editor.org/rfc/rfc5861

IETF HTTPAPI. The Idempotency-Key HTTP Header Field (rascunho).
datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header

IETF HTTPAPI. RateLimit header fields for HTTP (rascunho).
datatracker.ietf.org/doc/draft-ietf-httpapi-ratelimit-headers

MDN Web Docs. AbortController e AbortSignal.timeout().
developer.mozilla.org/docs/Web/API/AbortController

MDN Web Docs. Using server-sent events.
developer.mozilla.org/docs/Web/API/Server-sent_events/Using_server-sent_events

MDN Web Docs. Navigator.onLine e os eventos online e offline.
developer.mozilla.org/docs/Web/API/Navigator/onLine

MDN Web Docs. Intersection Observer API.
developer.mozilla.org/docs/Web/API/Intersection_Observer_API

web.dev. Cumulative Layout Shift (CLS).
web.dev/articles/cls

web.dev. Browser-level image lazy loading for the web.
web.dev/articles/browser-level-image-lazy-loading

Marc Brooker. Exponential Backoff And Jitter. AWS Architecture Blog, 2015.
aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter

AWS. Sharing objects with presigned URLs (Amazon S3).
docs.aws.amazon.com/AmazonS3/latest/userguide/ShareObjectPreSignedURL.html

PostgreSQL. NOTIFY e LISTEN.
postgresql.org/docs/current/sql-notify.html

TanStack Query. Documentação.
tanstack.com/query